



CSE471: System Analysis and Design
Project Report
Project Title: Calorify

Group No: 09, CSE471 Lab Section: 02, [Summer] [2026]	
ID	Name
24341140	Evan Masrur Jaber
23101261	Jarin Tasnim Dia
23101253	Mohammed Mashrekin Yakub
23301063	Noorani Faiza Khan

Submission Date: 05. 09. 2026

Table of Contents

1. System Request	3
Business need:	3
Business requirements:	3
Business value:	3
Special issues or constraints:	3
2. Functional Requirements	3
3. Technology (Framework, Languages)	3
4. Backend Development	3
5. User Interface Design	3
6. Frontend Development	3
7. User Manual	3
8. Performance and Network Analysis	3
9. Github Repo [Public] Link	4
10. Link of Deployed Project	4
11. Individual Contribution	4
12. References	5

1. System Request

Business need:

Bangladesh is experiencing a growing rise in diet related health issues, yet personalized nutrition guidance remains largely inaccessible to the average citizen. Professional dietitians are expensive and scarce, mainstream calorie tracking apps lack support for traditional Bangladeshi cuisine, and manual food logging is tedious enough that most users abandon it quickly. Calorify addresses this gap by providing an AI-powered, culturally aware nutrition and diet management platform that makes personalized dietary guidance affordable and accessible for the Bangladeshi population.

Business requirements:

Using Calorify, users will be able to efficiently manage their nutrition and dietary habits through a single integrated platform. The specific functionality the system should have includes the following:

- Allow users to register, log in, and maintain a personal health profile including age, weight, height, medical conditions, dietary preferences, and fitness goals.
- Enable AI powered calorie and macronutrient estimation from meal photographs and text based food entries.
- Generate personalized 7-day diet plans based on the user's health profile, uploaded medical lab reports, and wearable fitness data.
- Support uploading of medical and lab reports, with the system automatically extracting key health markers to inform diet plan recommendations.
- Allow users to connect their Google Health account to sync daily step count and active calories burned into the platform.
- Provide progress tracking through daily weight logs, plan adherence streaks, micro challenges, and milestone badges to sustain long term engagement.
- Automatically generate a weekly grocery shopping list from the user's active diet plan, with purchase links to local Bangladeshi online marketplaces.
- Send email notifications for meal reminders, weekly plan refreshes, and account events, and generate downloadable monthly health summary reports.
- Offer a premium subscription tier with advanced features, processed securely through the Bkash payment gateway.
- Provide an administrator dashboard with platform wide analytics including user growth, active users, diet plans generated, and calorie scans performed.

Business value:

We expect that Calorify will meaningfully improve the health and dietary habits of its users by delivering personalized, AI-generated meal plans that would otherwise require costly professional consultations. The platform's native support for Bangladeshi cuisine and local grocery integrations makes it immediately practical and relevant for its target audience. Features such as gamified challenges, streak tracking, and proactive health alerts are designed to build lasting user habits, driving long term retention. From a business perspective, the Bkash integrated subscription model provides a recurring revenue stream tailored to the local market, while the platform's cloud native architecture allows it to scale as its user base grows. With no existing solution in Bangladesh combining AI calorie estimation, medical report parsing, wearable integration, and local grocery links in one place, Calorify is positioned as a first mover in this space.

Special issues or constraints:

- Calorify's core AI features depend heavily on the Gemini API. Any changes to its pricing, availability, or response behavior will directly affect platform reliability and must be monitored closely.
- The backend is hosted on a free tier deployment, which may experience cold start delays that degrade the user experience for the first request after a period of inactivity.
- Lab reports vary widely in format and quality. The AI parser may produce incomplete or inconsistent results from poorly scanned or non standard documents, requiring fallback handling and user verification.
- Calorify collects and processes sensitive personal health information. This data must be stored securely and handled in accordance with applicable data privacy regulations, including Bangladesh's Digital Security Act.
- AI generated diet plans and nutritional analyses must be accompanied by a clear disclaimer that they do not substitute professional medical advice, particularly for users with diagnosed conditions.
- Production access to the Bkash Payment API requires formal merchant registration, which may delay the payment feature going live in a production environment.

2. Functional Requirements

- **Personalized Diet Plan Generator:** Users configure their health profile (age, weight, height, gender, activity level, goal, allergies) and select cuisine styles and dietary preferences. The backend automatically computes the user's daily calorie target using the Mifflin-St Jeor TDEE formula. It then aggregates up to three data sources: the health profile, any uploaded medical reports (diagnoses, HbA1c, allergies) and the latest wearable activity data (steps, calories burned) and bundles them into a

structured prompt sent to the Gemini API. Gemini returns a full 7-day meal plan (breakfast, lunch, snacks, dinner per day) with per meal calorie and macronutrient breakdowns. Users can also regenerate any individual day or single meal without affecting the rest of the plan.

- **AI Calorie Estimator (Image Upload):** Users upload a meal photo which is sent to a Gemini API. The model detects food items, estimates portion sizes and returns total calorie and macronutrient breakdowns. And logs the meal into a tracked food database.
- **AI Recipe Generator:** The Gemini API will also generate the recipe if the user clicks on the “Generate Recipe” button when viewing a specific item from the diet chart.
- **Daily Intake Tracker:** A daily summary view shows total calories consumed versus the user's daily calorie target with a visual progress bar for calories, carbs, protein, and fat. Weekly summaries are also aggregated.
- **Automated Wearable Data Ingestion:** Users connect their Google account via a Google OAuth2 consent flow initiated from the website. Once authorized, the backend exchanges the one time auth code for an access token and refresh token, which are stored in MongoDB against the user's profile. On each sync trigger the system fetches step count and calories burned using Google Health api using a fresh access token (auto refreshed via the stored refresh token). The returned daily step count and active calories burned are upserted into the user's Progress document in MongoDB, making the data available to the Diet Plan Generator and the Progress Tracker.
- **AI Meal Log:** Users can log meals by entering the food items they have eaten and the Gemini API will log the estimated calorie and micro nutrients of the food in the database or via the “AI Calorie Estimator(Image Upload)” feature
- **Progress Tracker:** Users log daily weight, calorie intake and plan adherence status. The system plots weight over time using a line chart, bar chart for calories and tracks a daily adherence streak. Progress data is linked to the calorie log and stored in MongoDB for use in the monthly health report.
- **Gamified Healthy Habit Challenges:** This feature encourages positive lifestyle changes by providing users with daily micro challenges (e.g., Drink 2L Water, Walk 5000 Steps, sleep 8 hrs, meditate 10 mins a day, Publish 1 diet post). Users track and log these activities; upon hitting the "Complete" button, the system updates their progress profile in MongoDB, calculating points and awarding corresponding points. Accumulating these points automatically unlocks tier-based milestones and visual badges (Hydration Pro, Sleep Guardian, Healthy started, Community diet pioneer etc) to enhance user engagement.
- **Ingredient Shopping List Generator:** Automatically derives a weekly grocery list from the user's active 7-day diet plan. All meal names across all 7 days are batched

into a single prompt sent to the Gemini API, which identifies the raw ingredients for each dish, consolidates duplicates, sums quantities across the week (assuming 2 servings per meal), and groups everything into standard categories (Proteins, Vegetables, Grains & Legumes, Dairy & Eggs, Spices & Condiments, Oils & Fats, Fruits, Other). The result is cached in MongoDB so subsequent visits return instantly without another API call. The list is automatically flagged as stale when any meal is regenerated, prompting a fresh Gemini call on the next refresh. Users can check off items as they shop, with checked state persisted per session.

- **Smart Medical Report Parser:** Users can upload a PDF or image scan of a lab report or diagnostic record. The file is preprocessed on the backend (images are downscaled and optimized) and then sent to the Gemini API using its multimodal file input capability. Gemini extracts a structured data set including confirmed diagnoses, HbA1c level, drug/food allergies, and a full blood marker panel (fasting glucose, cholesterol, LDL, HDL, triglycerides, blood pressure, hemoglobin, creatinine, liver enzymes, and any additional markers found). The extracted data is saved to MongoDB as a structured JSON object. When a new diet plan is generated, all past reports are merged diagnoses and allergies unioned across reports, HbA1c taken from the most recent and fed into the Gemini diet plan prompt to generate medically informed meal recommendations.
- **Monthly Health Report Generator:** At the end of each calendar month the system compiles the user's calorie data, average nutrient intake, weight trend, plan adherence percentage, and streak history into a structured report rendered as a downloadable PDF with charts and personalized suggestions.
- **Online Marketplace Integration:** Each ingredient in the shopping list has a "Buy Online" button that constructs a platform specific search URL using string interpolation and opens it in a new browser tab. Users select their preferred platform (Chaldal or Shopno) from a dropdown before browsing.
- **Pro Subscription & Payment Gateway:** A monetization module allowing users to upgrade to a premium tier for advanced features, such as unlimited AI food scans, unlimited bookmarks, smart medical report parser. This utilizes the Bkash Payment API to process secure checkout transactions. Upon successful payment, the system updates the user's subscription status in MongoDB and unlocks restricted platform capabilities.
- **Recipe Bookmark:** A bookmarking system that allows users to save specific AI generated recipes or meals they enjoyed from their weekly diet plans. By clicking a "Heart" icon, the recipe's JSON data is stored in a dedicated "bookmarks" array within the user's MongoDB profile, allowing them to browse a personalized library of their preferred Bangladeshi meals at any time.

- **Email Notification System:** The system sends email alerts for new user login, weekly plan refreshes, first blog post, challenge completion via the Gmail API. Users can manage notification preferences from their profile.
- **Community Diet Post & Suggestion Feed:** A community-driven module that allows users to write and publish diet blog posts, nutrition tips, and meal suggestions based on their personal interests and experiences. Users can provide detailed text content, and can upload an accompanying image (e.g., meal photos or infographics), like and comment on posts. Notification is sent to users getting comments and likes. The Node.js/Express backend processes the form payload and multipart image data, stores the post metadata (content, image URL/path, author ID, and timestamp) in MongoDB, and makes it available via a public community feed. Any registered user can browse, read, and explore these shared dietary insights to foster community engagement and healthy habits.

3. Technology (Framework, Languages)

- **Language:** Javascript
- **Framework:** React.js + Vite (Frontend), Node.js + Express.js (Backend)
- **Styling:** TailwindCSS
- **Database:** MongoDB
- **Deployment:** Vercel (Frontend), Render (Backend)

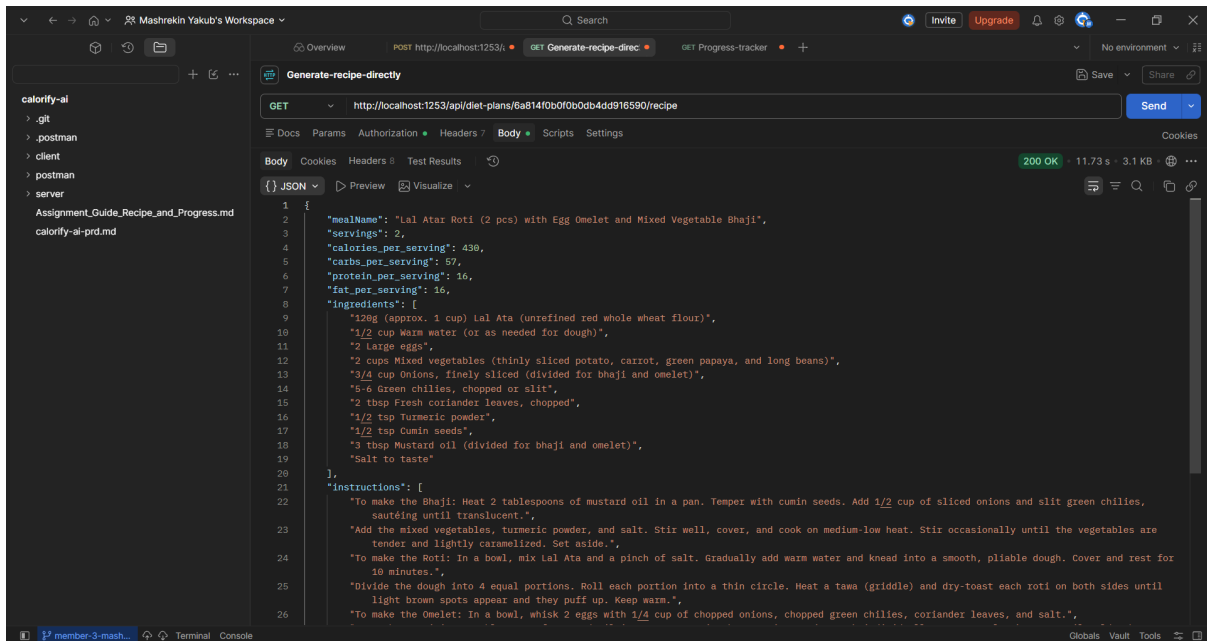
4. Backend Development

API-01

Feature - (Generate Recipe)

Description: This api generates recipe for a specific meal given the meal id

- Endpoint URL: `http://localhost:1253/api/diet-plans/:id/recipe`
- HTTP Method: Get
- Headers: Authorization: Bearer <token>
- Body: None



```

317 //
318 // @route GET /api/diet-plans/:id/recipe
319 // Generates or fetches cached recipe + trivia for a specific meal
320 //
321 const generateRecipe = async (req, res) => {
322   try {
323     const { id } = req.params;
324     const plan = await DietPlan.findOne({ user: req.user.id, isActive: true });
325     if (!plan) return res.status(404).json({ message: 'No active diet plan found' });
326
327     let targetMeal = null;
328     for (const day of plan.plan) {
329       const meal = day.meals.id(id);
330       if (meal) {
331         targetMeal = meal;
332         break;
333       }
334     }
335
336     if (!targetMeal) {
337       return res.status(404).json({ message: 'Meal not found in active diet plan' });
338     }
339
340     // Return cached recipe if already generated
341     if (targetMeal.recipe) {
342       try {
343         const parsed = JSON.parse(targetMeal.recipe);
344         return res.json({ mealName: targetMeal.name, ...parsed });
345       } catch (e) { /* Fall through to regenerate */ }
346     }
347
348     const prompt = `
349 Generate a detailed recipe, accurate and realistic nutritional values, and 2-3 localized health, nutritional, or historical facts (trivia) for the Bangladeshi food item "${targetMeal.name}".
350 The Food item is planned for ${targetMeal.meal}.
351
352 Provide the response in the following JSON format. Make sure to calculate the REALISTIC and ACCURATE nutritional values per serving and specify the servings count. DO NOT force the calories to hit an arbitrary target:
353 {
354   "servings": 2,
355   "calories_per_serving": 329,
356   "carbs_per_serving": 9,
357   "protein_per_serving": 17,
358   "fat_per_serving": 24,
359   "ingredients": ["ingredient 1 with quantity", "ingredient 2 with quantity"],
360   "instructions": ["step 1", "step 2"],
361   "trivia": ["fact 1 about ingredients in a Bangladeshi diet", "fact 2 about ingredients..."]
362 }
363
364 Return ONLY valid raw JSON. Do not include markdown code block characters like \`\`\`json.
365 `;
366
367 const rawResponse = await generateText(prompt);
368 const cleanJsonStr = rawResponse.replace(/`json`'/g, '').trim();
369 const parsedData = JSON.parse(cleanJsonStr);
370
371 parsedData.image_url = await generateDishImageUrl(targetMeal.name);
372
373 targetMeal.recipe = JSON.stringify(parsedData);
374 await plan.save();
375
376 res.json({ mealName: targetMeal.name, ...parsedData });
377 } catch (error) {
378   res.status(500).json({ message: error.message });
379 }
380 };

```

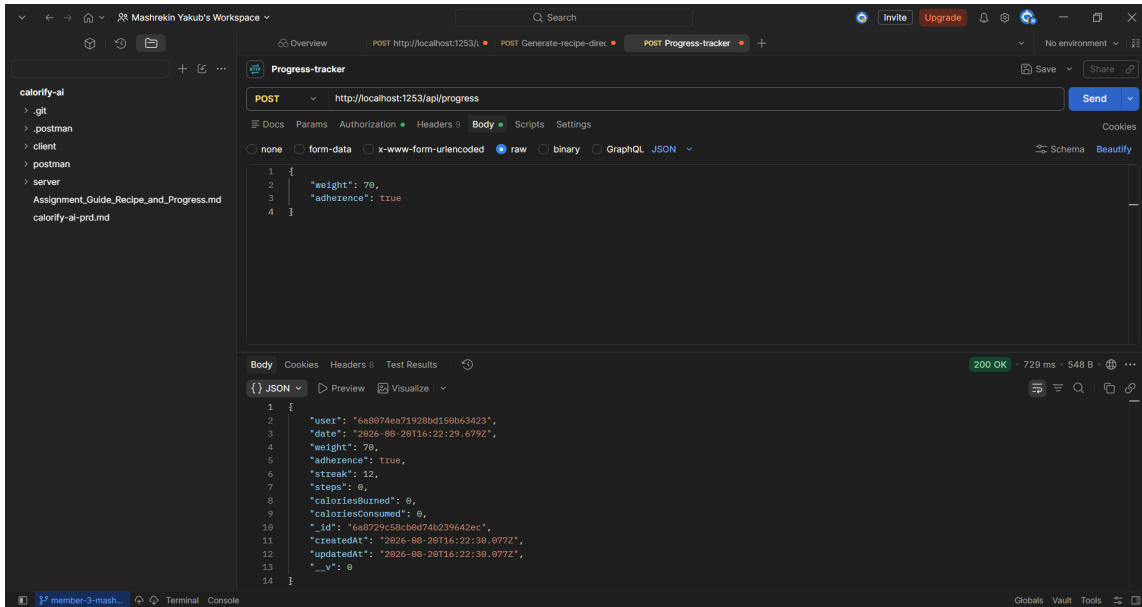
API-02

Feature - (Progress Tracker)

Description: This api logs weight of the day to the database given the weight and adherence state

- Endpoint URL: `http://localhost:1253/api/progress`
- HTTP Method: Post

- Headers: Authorization: Bearer <token>, Content-Type: application/json
- Body: {
"weight": 70,
"adherence": true
}



```

4 // Route POST /api/progress
5 const logProgress = async (req, res) => {
6   try {
7     const { weight, adherence } = req.body;
8
9     // Get start and end of today
10    const startOfDay = new Date();
11    startOfDay.setHours(0, 0, 0, 0);
12    const endOfDay = new Date();
13    endOfDay.setHours(23, 59, 59, 999);
14
15    // Get yesterday's boundaries for streak calculation
16    const startOfYesterday = new Date(startOfDay);
17    startOfYesterday.setDate(startOfYesterday.getDate() - 1);
18    const endOfYesterday = new Date(endOfDay);
19    endOfYesterday.setDate(endOfYesterday.getDate() - 1);
20
21    // Find yesterday's entry to calculate the streak
22    const yesterdayEntry = await Progress.findOne({
23      user: req.user.id,
24      date: { $gte: startOfYesterday, $lte: endOfYesterday }
25    });
26
27    let newStreak = 0;
28    if (adherence) {
29      newStreak = (yesterdayEntry ? yesterdayEntry.streak : 0) + 1;
30    } else {
31      newStreak = 0;
32    }
33
34    // Insert today's entry
35    let todayEntry = await Progress.findOne({
36      user: req.user.id,
37      date: { $gte: startOfDay, $lte: endOfDay }
38    });
39
40    // Calculate calories consumed today from Meallogs
41    const meallogsToday = await Meallog.find({
42      user: req.user.id,
43      date: { $gte: startOfDay, $lte: endOfDay }
44    });
45    const caloriesConsumed = meallogsToday.reduce((acc, log) => acc + (log.calories || 0), 0);
46
47    if (todayEntry) {
48      if (weight !== undefined) todayEntry.weight = weight;
49      if (adherence !== undefined) todayEntry.adherence = adherence;
50      todayEntry.streak = newStreak;
51      todayEntry.caloriesConsumed = caloriesConsumed;
52      await todayEntry.save();
53    } else {
54      todayEntry = await Progress.create({
55        user: req.user.id,
56        date: new Date(), // store exact time
57        weight,
58        adherence,
59        streak: newStreak,
60        caloriesConsumed
61      });
62    }
63
64    res.status(200).json(todayEntry);
65  } catch (error) {
66    res.status(500).json({ message: error.message });
67  }
68 };

```

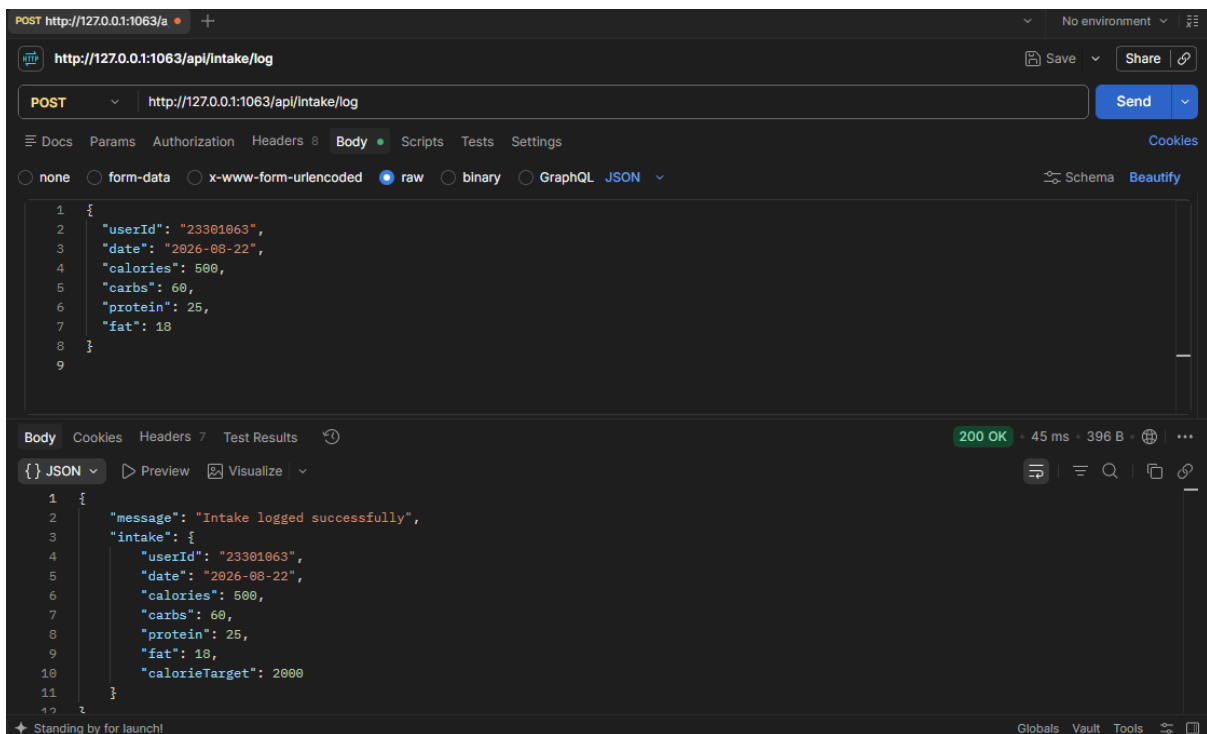
API-03

Feature - (Daily Intake Tracker)

Description: This api logs a meal given its calories and micronutrients values

- Endpoint URL: 127.0.0.1:1063/api/intake/log
- HTTP Method: POST
- Headers: Content-Type: application/json
- Body: {"userId": "23301063", "date": "2026-08-22", "calories": 500, "carbs": 60, "protein": 25, "fat": 18}

```
app.post('/api/intake/log', (req, res) => {  
  const { userId, date, calories, carbs, protein, fat } = req.body;  
  const newEntry = { userId, date, calories, carbs, protein, fat, calorieTarget: 2000 };  
  intakeLogs.push(newEntry);  
  res.status(200).json({ message: "Intake logged successfully", intake: newEntry });  
});
```



API-04

Feature - (Automated Wearable Data Ingestion)

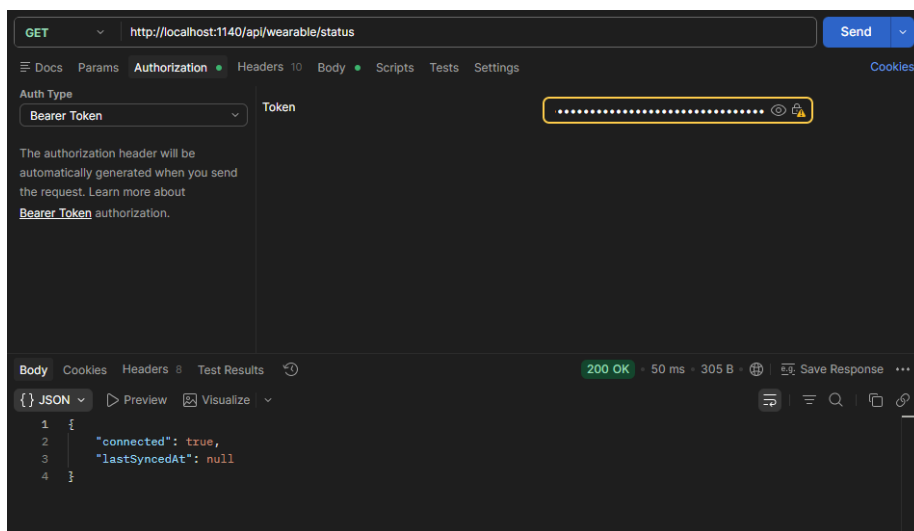
Description: This api checks the google account connection status and last sync time

- Endpoint URL: `http://localhost:1140/api/wearable/status`
- HTTP Method: Get
- Headers: None
- Body: {}

```
const getStatus = async (req, res) => {
  try {
    const user = await User.findById(req.user.id).select('googleFit');

    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    res.json({
      connected: user.googleFit?.connected || false,
      lastSyncedAt: user.googleFit?.lastSyncedAt || null,
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```



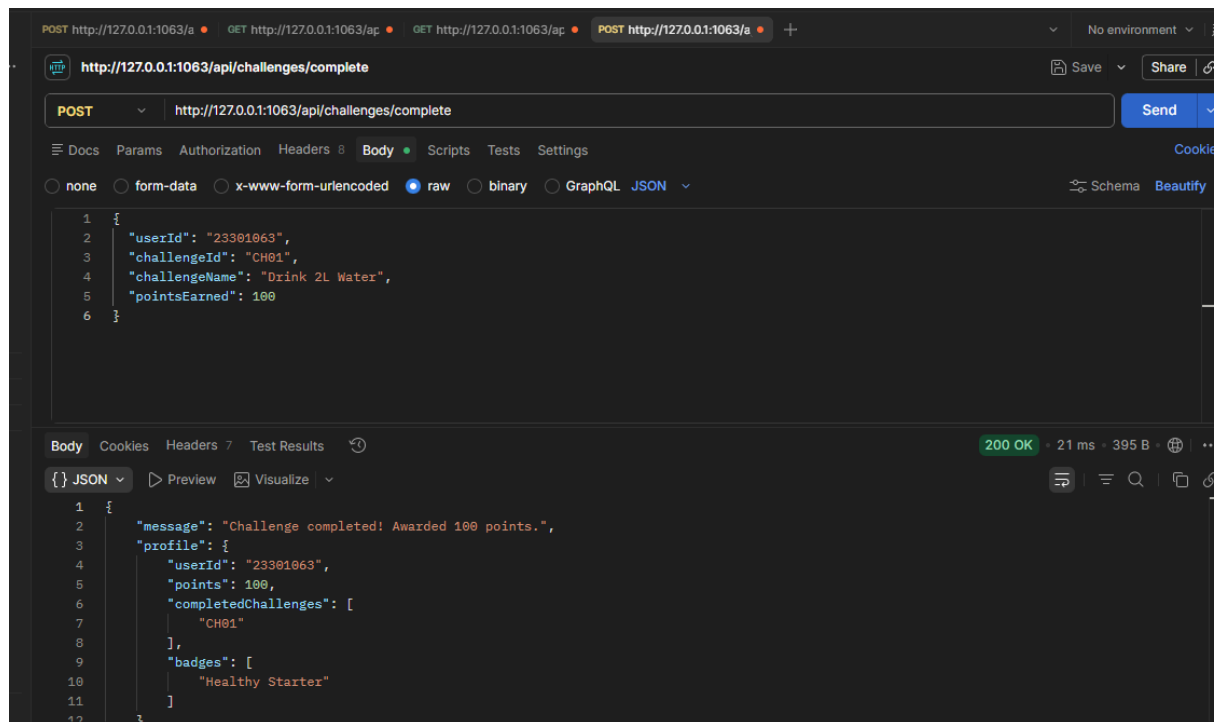
API-05

Feature - (Gamified Healthy Habit Challenges)

Description: This api marks a daily challenge as completed given the challenge information

- Endpoint URL: 127.0.0.1:1063/api/challenges/complete
- HTTP Method: POST
- Headers: Content-Type: application/json
- Body: {"userId": "23301063", "challengeId": "CH01", "challengeName": "Drink 2L Water", "pointsEarned": 100}

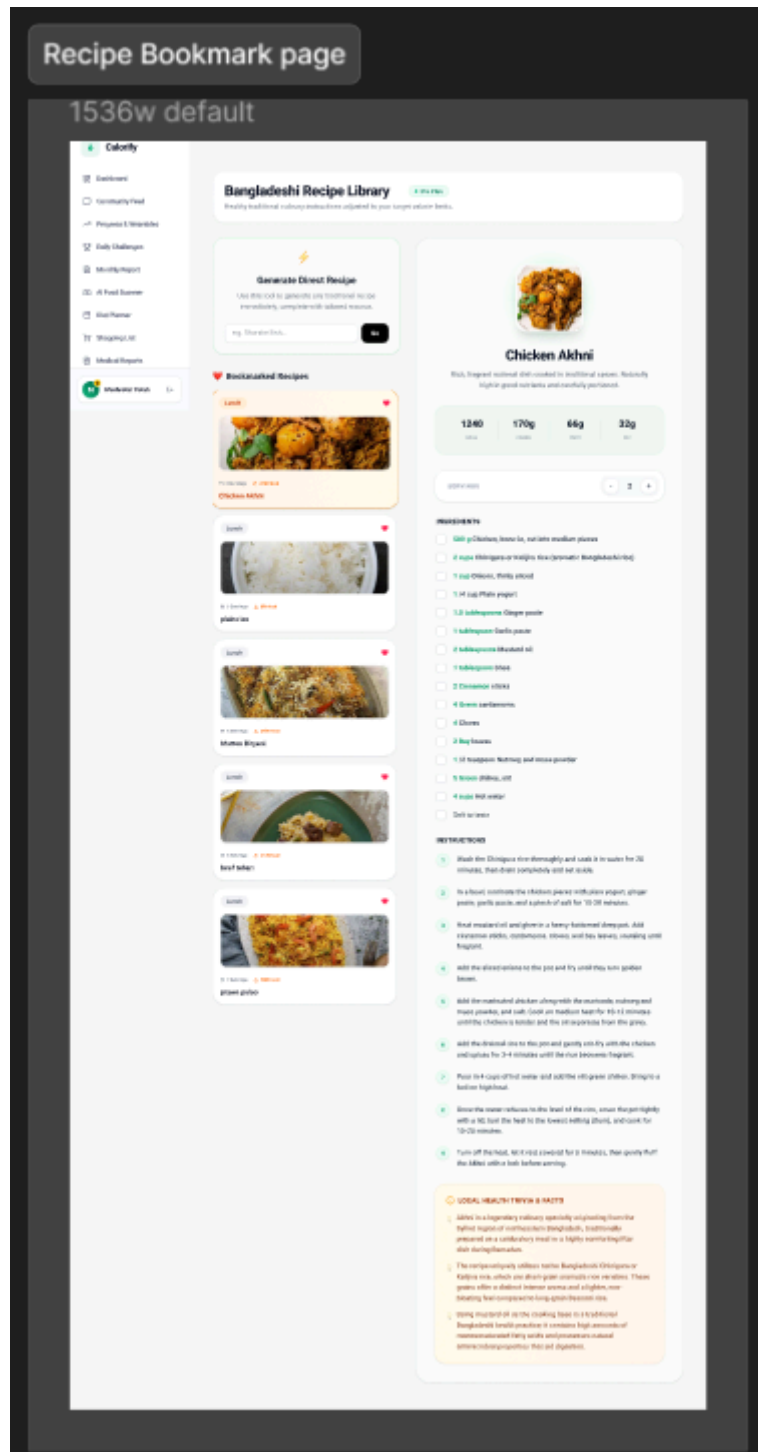
```
app.post('/api/challenges/complete', (req, res) => {
  const { userId, challengeId, challengeName, pointsEarned } = req.body;
  if (!userChallenges[userId]) userChallenges[userId] = { userId, points: 0, completedChallenges: [], badges: [] };
  const profile = userChallenges[userId];
  profile.points += pointsEarned;
  if (!profile.completedChallenges.includes(challengeId)) profile.completedChallenges.push(challengeId);
  if (profile.points >= 100 && !profile.badges.includes("Healthy Starter")) profile.badges.push("Healthy Starter");
  res.status(200).json({ message: `Challenge completed! Awarded ${pointsEarned} points.`, profile });
});
```



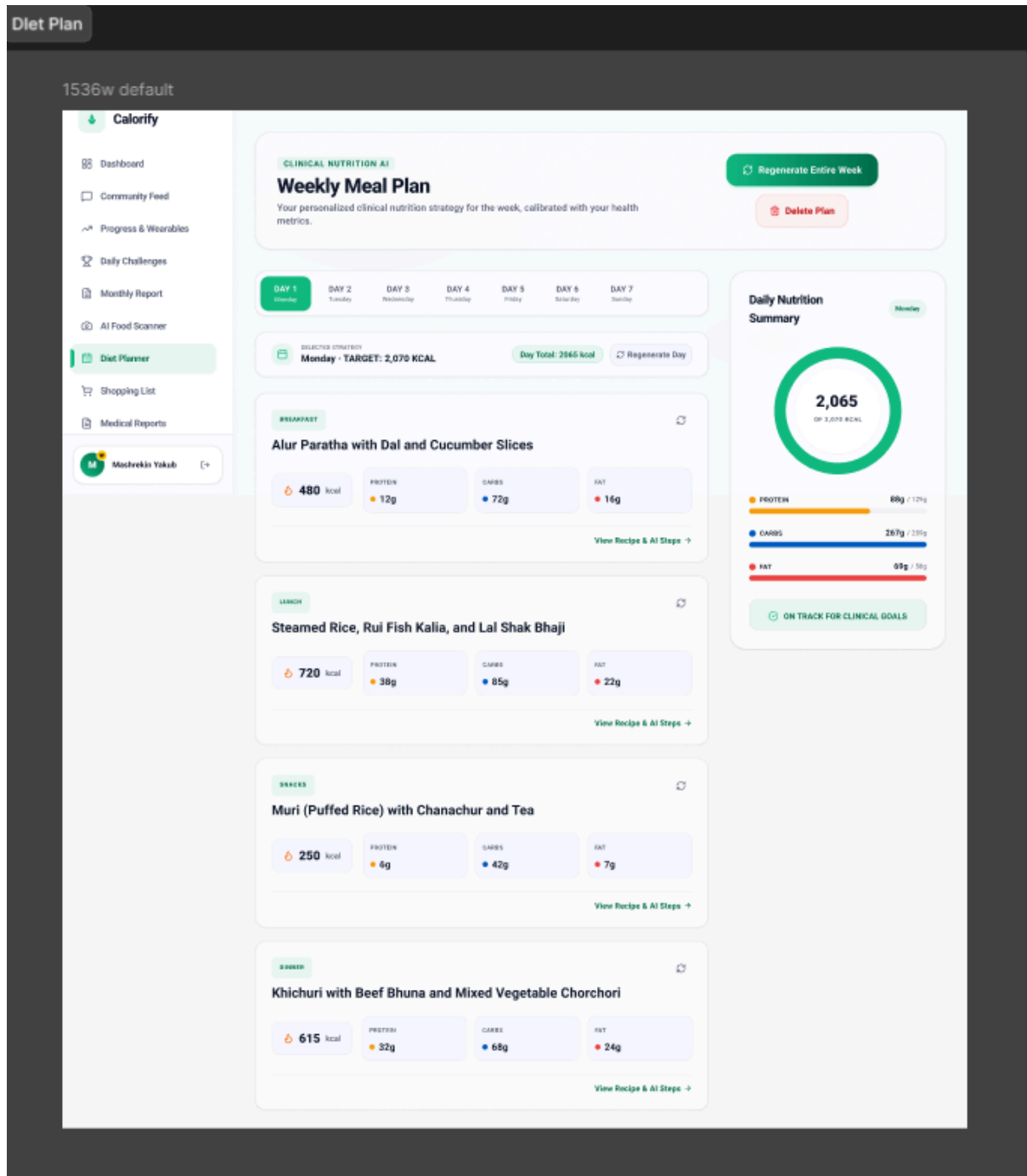
5. User Interface Design

Figma Project Link: [Calorify Design](#)

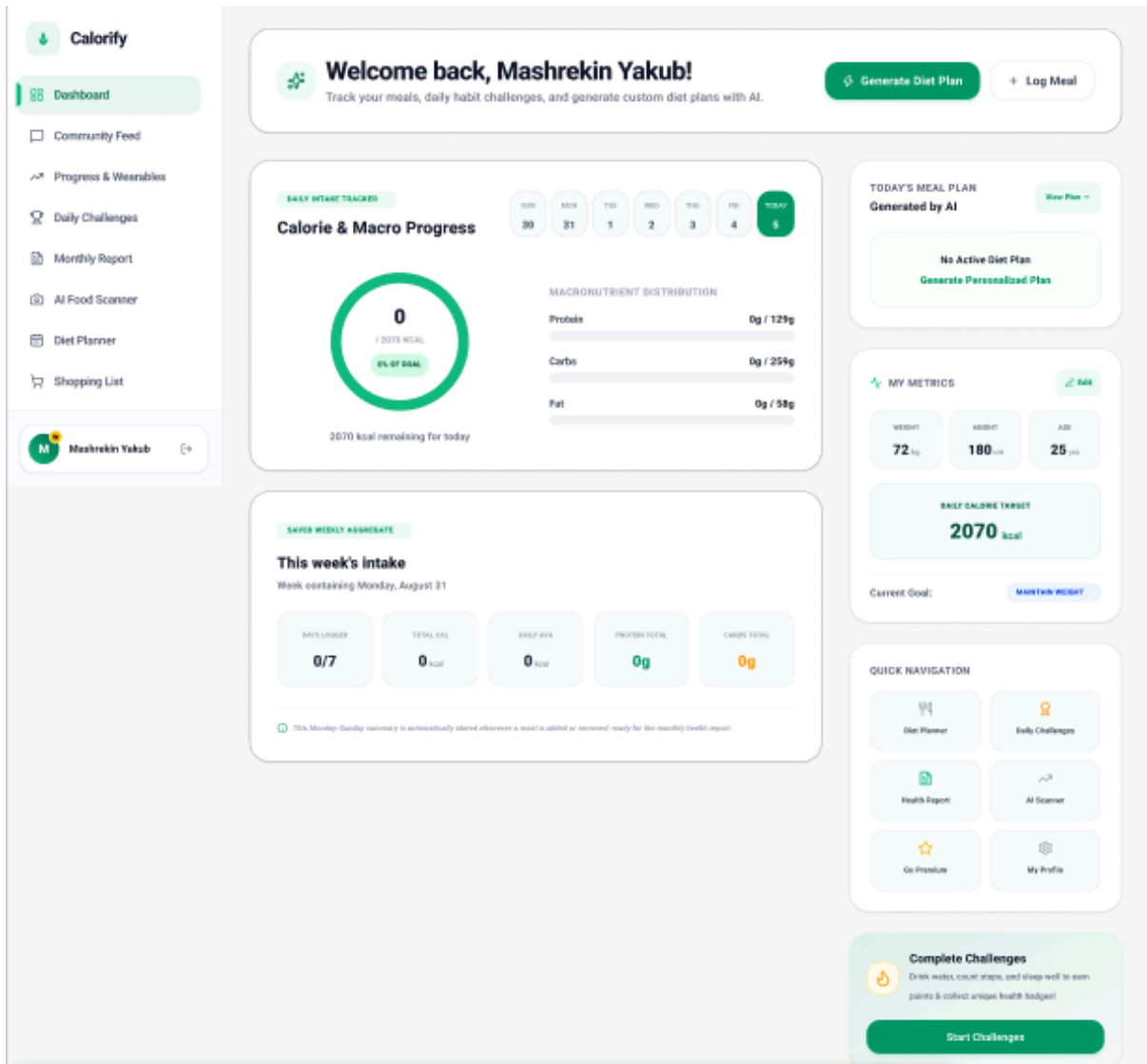
Interface Design-01 (Recipe Bookmark Page)



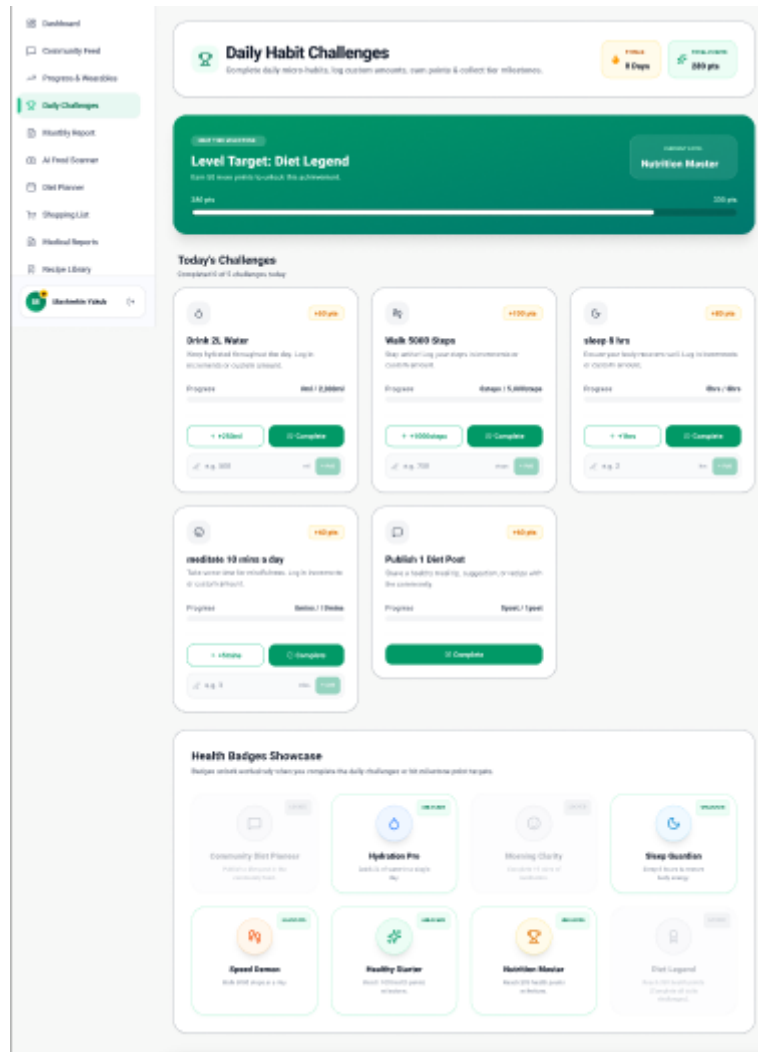
Interface Design-02 (Diet Plan Page)



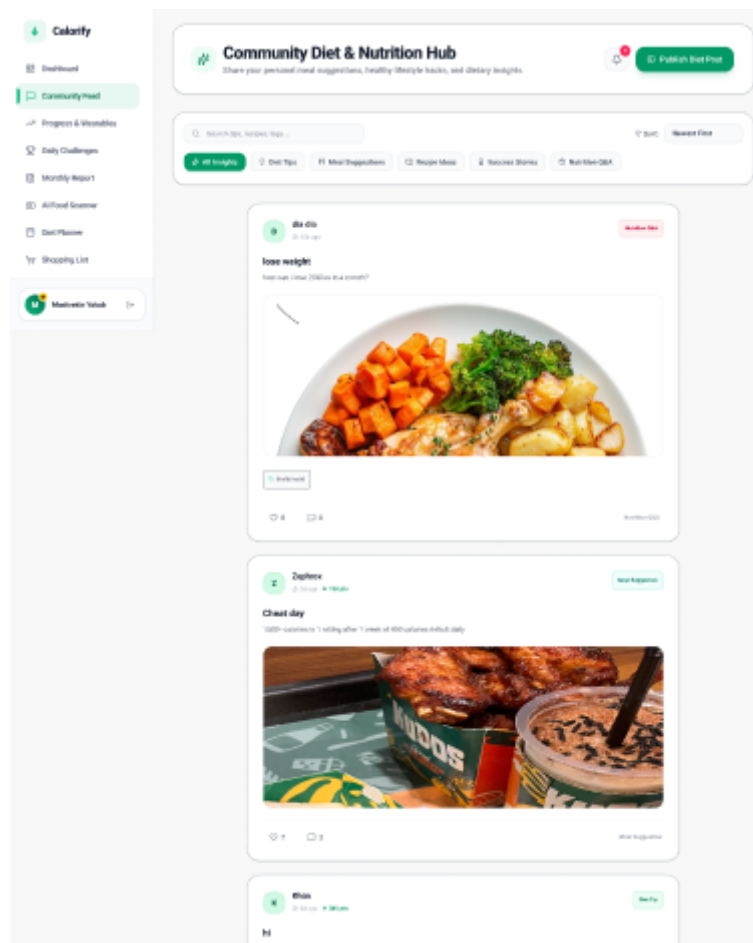
Interface Design-03 (Dashboard Page)



Interface Design-04 (Daily Habit Challenge Page)



Interface Design-05 (Community Feed Page)



6. Frontend Development

Diet Plan Feature

Lines 1–111 of DietPlan.jsx establish the foundation of the Diet Plan page. The file begins by importing React hooks (useState, useEffect, useMemo), all diet plan API service functions, Lucide icons, and an animated WebGL background component. Two static arrays, CUISINE_PRESETS and DIET_PREFERENCE_PRESETS define the selectable cuisine styles and dietary filters shown in the preferences modal. Inside the component, all UI state is declared including the active plan, loading flags, a unified prefModal object that tracks generation scope (week, day, or meal), and recipe/bookmark states. On mount, a useEffect simultaneously fetches the user's existing active diet plan and a generation context preview (profile completeness, lab reports, wearable data), rendering either the weekly meal grid or the Generate CTA screen depending on whether a plan already exists.

```

client > src > pages > DietPlan > DietPlan.jsx > ...
1 import React, { useState, useEffect, useMemo } from 'react';
2 import { createPortal } from 'react-dom';
3 import {
4   getActiveDietPlan,
5   generateDietPlan,
6   regenerateDay,
7   regenerateMeal,
8   generateRecipe,
9   getGenerationContext,
10  deleteActiveDietPlan,
11 } from '../services/dietPlanService';
12 import { addBookmark, getBookmarks } from '../services/bookmarkService';
13 import {
14   Sparkles,
15   RefreshCw,
16   Calendar,
17   Flame,
18   Utensils,
19   Info,
20   Heart,
21   CheckCircle,
22   ArrowRight,
23   AlertTriangle,
24   X,
25   ChefHat,
26   Globe,
27   Salad,
28   MessageSquare,
29   Check,
30   Trash2,
31 } from 'lucide-react';
32 import BgShader from '../components/BgShader';
33
34 const CUISINE_PRESETS = [
35   { id: 'bangladeshi', name: 'Bangladeshi / Bengali', emoji: '🍛', desc: 'Authentic local home dishes (Shak, Ilish, Daal, Bharta)' },
36   { id: 'pan_asian', name: 'Pan-Asian', emoji: '🍜', desc: 'Thai, Japanese, Chinese stir-fries, steamed fish, soups' },
37   { id: 'continental', name: 'Continental / Western', emoji: '🍖', desc: 'Grilled proteins, baked fish, roasted veggies, salads' },
38   { id: 'italian', name: 'Italian', emoji: '🍝', desc: 'Whole-wheat pasta, marinara, grilled chicken, minestrone' },
39   { id: 'mediterranean', name: 'Mediterranean', emoji: '🥗', desc: 'Greek salads, hummus, grilled seafood, olive oil dressings' },
40   { id: 'middle_eastern', name: 'Middle Eastern', emoji: '🍲', desc: 'Shish tawook, lentil shorba, baked kababs, spiced bowls' },
41   { id: 'mexican', name: 'Mexican / Latin', emoji: '🌮', desc: 'Burrito bowls, fajitas, black beans, guacamole, tacos' },
42   { id: 'indian', name: 'Pan-Indian', emoji: '🍛', desc: 'Tandoori proteins, dal tadka, palak, idli, aromatic curries' },
43 ];
44
45 const DIET_PREFERENCE_PRESETS = [
46   { id: 'diabetic_friendly', name: 'Diabetic-Friendly (Low GI)', emoji: '🍓', desc: 'Complex carbs, high fiber, zero refined sugars' },
47   { id: 'low_carb_keto', name: 'Low-Carb / Keto', emoji: '🥑', desc: 'Minimized rice/flour; focuses on lean proteins & greens' },
48   { id: 'high_protein', name: 'High-Protein', emoji: '🍖', desc: 'Maximized protein density for muscle repair & satiety' },
49   { id: 'low_oil', name: 'Low-Oil / Heart-Healthy', emoji: '🥑', desc: 'Steamed, boiled, air-fried with minimal cooking fats' },
50   { id: 'vegetarian_vegan', name: 'Vegetarian / Plant-Based', emoji: '🌱', desc: '100% plant proteins: paneer, lentils, seasonal veggies' },
51   { id: 'gluten_free', name: 'Gluten-Free', emoji: '🌾', desc: 'Naturally gluten-free grains, vegetables, and proteins' },
52   { id: 'quick_prep', name: 'Quick & Easy (<20 mins)', emoji: '⚡', desc: 'Fast preparation times with accessible ingredients' },
53   { id: 'balanced_macro', name: 'Balanced Macro', emoji: '🍌', desc: 'Balanced carbs, protein, and healthy fats' },
54 ];
55

```

```

56 export default function DietPlan() {
57   const [activePlan, setActivePlan] = useState(null);
58   const [loading, setLoading] = useState(true);
59   const [generating, setGenerating] = useState(false);
60
61   // Selected Day tab (0 to 6)
62   const [selectedDayIndex, setSelectedDayIndex] = useState(0);
63
64   // Cuisine & Preferences Modal State
65   const [prefModal, setPrefModal] = useState({
66     isOpen: false,
67     scope: 'week', // 'week' | 'day' | 'meal'
68     targetDayIndex: 0,
69     targetDayName: '',
70     targetMeal: null,
71     cuisines: ['bangladeshi'],
72     customCuisine: '',
73     dietPreferences: [],
74     customDietPreference: '',
75     customNotes: '',
76   });
77
78   // Recipe Modal states
79   const [selectedMeal, setSelectedMeal] = useState(null);
80   const [recipeLoading, setRecipeLoading] = useState(false);
81   const [recipeData, setRecipeData] = useState(null);
82   const [recipeError, setRecipeError] = useState('');
83   const [bookmarking, setBookmarking] = useState(false);
84   const [bookmarked, setBookmarked] = useState(false);
85
86   // Pre-generation context state
87   const [genContext, setGenContext] = useState(null);
88   const [contextLoading, setContextLoading] = useState(true);
89
90   // Delete Plan Modal State
91   const [showDeleteConfirm, setShowDeleteConfirm] = useState(false);
92   const [deletingPlan, setDeletingPlan] = useState(false);
93   const [deleteError, setDeleteError] = useState('');
94
95   useEffect(() => {
96     fetchPlan();
97     fetchContext();
98   }, []);
99
100   const fetchPlan = async () => {
101     try {
102       setLoading(true);
103       const { data } = await getActiveDietPlan();
104       setActivePlan(data);
105     } catch (err) {
106       console.error(err);
107       setActivePlan(null);
108     } finally {
109       setLoading(false);
110     }
111   };

```

Pro Subscription Feature

The frontend subscription module manages the client side lifecycle of a bKash payment transaction in three stages. First, *handleBkashPay* calls *initiateSubscription*, a thin Axios wrapper that requests a payment URL from the backend (POST /subscriptions/initiate); on success, the browser is redirected to bKash's hosted checkout page, keeping card and payment credentials off the application's own frontend. Second, once bKash completes the transaction, it redirects the user back to /subscription with query parameters (status, paymentID) indicating the outcome; a *useEffect* hook parses these parameters on mount to detect success, failure, or cancellation. Third, on a successful redirect, *verifyPayment* sends the *paymentID* to the backend's callback endpoint (POST /subscriptions/callback) for

server-side confirmation against bKash before trusting the result - the frontend never marks a user as Pro purely from the redirect status, closing the gap where a user could manually craft a "success" URL to fake payment. Once verified, the component updates the shared AuthContext (login({ ...user, isPro: true }, token)), which immediately unlocks Pro-gated UI elsewhere in the app (unlimited AI food scans, unlimited bookmarks, the medical report parser) without requiring a full re-login.

```

client > src > pages > Subscription > Subscription.jsx > Subscription
1 import React, { useState, useEffect } from 'react';
2 import { useLocation, useNavigate } from 'react-router-dom';
3 import { useAuth } from '../../../context/AuthContext';
4 import { initiateSubscription, getSubscriptionStatus } from '../../../services/subscriptionService';
5 import api from '../../../services/api'; // For the execute callback
6 import { Check, Zap, Shield, Camera, ArrowRight, Loader, Bookmark, FileText } from 'lucide-react';
7
8 export default function Subscription() {
9   const { user, login, token } = useAuth();
10   const location = useLocation();
11   const navigate = useNavigate();
12
13   const [loading, setLoading] = useState(false);
14   const [verifying, setVerifying] = useState(false);
15   const [success, setSuccess] = useState(false);
16   const [errorMsg, setErrorMsg] = useState('');
17
18   // Check for bKash callback params on mount
19   useEffect(() => {
20     const params = new URLSearchParams(location.search);
21     const bkashStatuses = params.getAll('status');
22     const paymentID = params.get('paymentID');
23
24     const isSuccess = bkashStatuses.includes('success');
25     const isFailure = bkashStatuses.includes('failure') || bkashStatuses.includes('cancel');
26
27     if (isSuccess && paymentID) {
28       verifyPayment(paymentID);
29     } else if (isFailure) {
30       setErrorMsg('Payment failed or cancelled. Please try again.');
```

```

31       // Clear URL
32       window.history.replaceState({}, document.title, '/subscription');
33     }
34   }, [location]);
35
36   const verifyPayment = async (paymentID) => {
37     try {
38       setVerifying(true);
39       const { data } = await api.post('/subscriptions/callback', {
40         paymentID,
41         status: 'success'
42       });
43       if (data.success) {
44         setSuccess(true);
45         if (user && token) {
46           login({ ...user, isPro: true }, token);
47         }
48         // Clean URL
49         window.history.replaceState({}, document.title, '/subscription');
50       }
51     } catch (err) {
52       console.error('Error verifying payment:', err);
53       setErrorMsg(err.response?.data?.message || 'Failed to verify payment.');
```

```

54     } finally {
55       setVerifying(false);
56     }
57   };
58
59   const handleBkashPay = async () => {
60     try {
61       setLoading(true);
62       setErrorMsg('');
63       const { data } = await initiateSubscription('pro');
64       if (data.paymentURL) {
65         window.location.href = data.paymentURL;
66       } else {
67         throw new Error('No payment URL received');
```


Daily Habit Challenge Feature

The frontend for the **Daily Habit Challenges** feature is built in React and Tailwind CSS at client/src/pages/Challenges/Challenges.jsx, delivering an interactive gamified wellness dashboard where users track 5 core daily micro-habits: water intake, steps walked, sleep duration, meditation, and community diet posting. It features responsive habit cards equipped with dynamic progress bars, quick-increment step buttons (+Step), custom numeric logging inputs, and one-tap completion triggers, all integrated with a real-time points engine (0–330 pts, streak counters, multi-tier milestone level bars (*Healthy Starter*, *Nutrition Master*, *Diet Legend*), and a visual health badge showcase that unlocks achievement badges with celebratory confetti animations via asynchronous API calls in client/src/services/challengeService.js.

```
client > src > pages > Challenges > Challenges.jsx > ...
You, 2 days ago | 1 author (You)
You, last week • Member4Features

1 import React, { useState, useEffect } from 'react';
2 import { useAuth } from '../../../context/AuthContext';
3 import { getUserProfile } from '../../../services/userService';
4 import { getProgress } from '../../../services/progressService';
5 import { getTodayChallenges, logChallengeProgress, completeChallenge } from '../../../services/challengeService';
6 import {
7   Flame,
8   Target,
9   Trophy,
10  Award,
11  Zap,
12  Sparkles,
13  Droplet,
14  Moon,
15  Smile,
16  Dumbbell,
17  Footprints,
18  CheckCircle,
19  HelpCircle,
20  Plus,
21  MessageSquare,
22  Edit3,
23 } from 'lucide-react';
24
25 const badgesConfig = [
26   {
27     key: 'community_diet_pioneer',
28     name: 'Community Diet Pioneer',
29     description: 'Publish a diet post in the community feed.',
30     icon: MessageSquare,
31     color: 'from-teal-400 to-emerald-600 text-teal-600 bg-teal-50 border-teal-200'
32   },
33   {
34     key: 'hydration_pro',
35     name: 'Hydration Pro',
36     description: 'Drink 2L of water in a single day.',
37     icon: Droplet,
38     color: 'from-blue-400 to-indigo-500 text-blue-500 bg-blue-50 border-blue-200'
39   },
40   {
41     key: 'morning_clarity',
42     name: 'Morning Clarity',
43     description: 'Complete 10 mins of meditation.',
44     icon: Smile,
45     color: 'from-purple-400 to-pink-500 text-purple-500 bg-purple-50 border-purple-200'
46   },
47   {
48     key: 'sleep_guardian',
49     name: 'Sleep Guardian',
50     description: 'Sleep 8 hours to restore body energy.',
51     icon: Moon,
52     color: 'from-sky-400 to-blue-600 text-sky-600 bg-sky-50 border-sky-200'
53   },
54   {
55     key: 'speed_demon',
56     name: 'Speed Demon',
57     description: 'Walk 5000 steps in a day.'
```

```

25 const badgesConfig = [
26   {
27     icon: Footprints,
28     color: 'from-orange-400 to-red-500 text-orange-500 bg-orange-50 border-orange-200'
29   },
30   {
31     key: 'healthy_starter',
32     name: 'Healthy Starter',
33     description: 'Reach 100 health points milestone.',
34     icon: Sparkles,
35     color: 'from-emerald-400 to-teal-500 text-emerald-500 bg-emerald-50 border-emerald-200'
36   },
37   {
38     key: 'nutrition_master',
39     name: 'Nutrition Master',
40     description: 'Reach 200 health points milestone.',
41     icon: Trophy,
42     color: 'from-amber-400 to-orange-500 text-amber-500 bg-amber-50 border-amber-200'
43   },
44   {
45     key: 'diet_legend',
46     name: 'Diet Legend',
47     description: 'Reach 330 health points (Complete all daily challenges).',
48     icon: Award,
49     color: 'from-yellow-400 to-amber-500 text-yellow-500 bg-yellow-50 border-yellow-200'
50   }
51 ];
52
53 export default function Challenges() {
54   const { login } = useAuth();
55
56   // States
57   const [profile, setProfile] = useState(null);
58   const [challenges, setChallenges] = useState([]);
59   const [currentStreak, setCurrentStreak] = useState(0);
60   const [loading, setLoading] = useState(true);
61   const [actionLoading, setActionLoading] = useState(false);
62   const [customInputs, setCustomInputs] = useState({});
63   const [showConfetti, setShowConfetti] = useState(false);
64   const [unlockedMessage, setUnlockedMessage] = useState('');
65
66   // Fetch all details on mount
67   const fetchData = async () => {
68     try {
69       setLoading(true);
70       const localDataStr = new Date().toLocaleDateString('en-CA');
71       const [profileRes, challengesRes, progressRes] = await Promise.allSettled([
72         getProfile(),
73         getTodayChallenges(localDataStr),
74         getProgress()
75       ]);
76
77       if (profileRes.status === 'fulfilled') {
78         setProfile(profileRes.value.data);
79         const token = localStorage.getItem('calorify_token');
80         login(profileRes.value.data, token);
81       }
82
83       if (challengesRes.status === 'fulfilled') {
84         setChallenges(challengesRes.value.data || []);
85       }
86
87       if (progressRes.status === 'fulfilled') {
88         const data = progressRes.value.data || [];
89         if (data.length > 0) {
90           setCurrentStreak(data[data.length - 1].streak || 0);
91         }
92       }
93     } catch (err) {
94       console.error('Error fetching challenges data:', err);
95     } finally {
96       setLoading(false);
97     }
98   };
99
100   useEffect(() => {
101     fetchData();
102   }, []);
103
104   // Handle logging incremental progress (+Step or custom manual amount)
105   const handleLogProgress = async (id, customAmount = null) => {
106     try {
107       setActionLoading(true);
108       const amountToSend = customAmount ? Number(customAmount) : undefined;
109       const { data } = await logChallengeProgress(id, amountToSend);
110     }

```

```

client > src > pages > Challenges > Challenges.jsx
134 export default function Challenges() {
135   const handleLogProgress = async (id, customAmount = null) => {
136     const { data } = await logChallengeProgress(id, amountToSend);
137
138     // Refresh list
139     setChallenges(prev => prev.map(c => c._id === id ? data.challenge : c));
140
141     // Clear custom input
142     setCustomInputs(prev => ({ ...prev, [id]: '' }));
143
144     // Re-fetch profile for updated points and unlocked badges
145     const profilesRes = await getProfile();
146     setProfile(profilesRes.data);
147     const token = localStorage.getItem('calorify_token');
148     login(profilesRes.data, token);
149
150     // Check if badge unlocked
151     if (data.unlockedBadge) {
152       const badgeObj = badgesConfig.find(b => b.key === data.unlockedBadge);
153       if (badgeObj) {
154         setUnlockedMessage('🎉 Congratulations! You unlocked the badge "${badgeObj.name}"!');
155         setShowConfetti(true);
156         setTimeout(() => {
157           setShowConfetti(false);
158           setUnlockedMessage('');
159         }, 6000);
160       }
161     }
162   } catch (err) {
163     console.error('Error logging challenge progress:', err);
164   } finally {
165     setActionLoading(prev => ({ ...prev, [id]: false }));
166   }
167 }
168
169 // Handle 100% full completion in one tap
170 const handleCompleteChallenge = async (id) => {
171   try {
172     setActionLoading(prev => ({ ...prev, [id]: true }));
173     const { data } = await completeChallenge(id);
174
175     // Refresh list
176     setChallenges(prev => prev.map(c => c._id === id ? data.challenge : c));
177
178     // Re-fetch profile
179     const profilesRes = await getProfile();
180     setProfile(profilesRes.data);
181     const token = localStorage.getItem('calorify_token');
182     login(profilesRes.data, token);
183
184     // Confetti & announcement
185     if (data.unlockedBadge) {
186       const badgeObj = badgesConfig.find(b => b.key === data.unlockedBadge);
187       if (badgeObj) {
188         setUnlockedMessage('🎉 Congratulations! You unlocked the badge "${badgeObj.name}"!');
189         setShowConfetti(true);
190         setTimeout(() => {
191           setShowConfetti(false);
192           setUnlockedMessage('');
193         }, 6000);
194       }
195     }
196   } catch (err) {
197     console.error('Error completing challenge:', err);
198   } finally {
199     setActionLoading(prev => ({ ...prev, [id]: false }));
200   }
201 }
202
203 // Calculate user points earned from today's daily challenges (starts at 0 on new day)
204 const todayPoints = challenges.reduce(
205   (sum, c) => (c.completed ? sum + (c.pointsReward || 0) : sum),
206   0
207 );
208 const userPoints = todayPoints;
209 const userUnlockedBadges = profile?.unlockedBadges || [];
210
211 // Calculate next milestone progress based on today's points earned
212 const getNextMilestoneDetails = () => {
213   if (userPoints < 100) {
214     return {
215       nextName: 'Healthy Starter',
216       targetPoints: 100,
217       pct: Math.min(Math.round((userPoints / 100) * 100), 100),
218       pointsLeft: Math.max(100 - userPoints, 0)
219     };
220   }
221 }

```

```

O src X Challenges.jsx
class > src > pages > Challenges > @ Challenges.jsx > _
84 export default function Challenges() {
216 const getMilestoneDetails = () => {
222   pointsLeft: Math.max(100 - userPoints, 0)
223 };
224 } else if (userPoints < 200) {
225   return {
226     nextName: 'Nutrition Master',
227     targetPoints: 200,
228     pct: Math.min(Math.round((userPoints / 200) * 100), 100),
229     pointsLeft: Math.max(200 - userPoints, 0)
230 };
231 } else if (userPoints < 330) {
232   return {
233     nextName: 'Diet Legend',
234     targetPoints: 330,
235     pct: Math.min(Math.round((userPoints / 330) * 100), 100),
236     pointsLeft: Math.max(330 - userPoints, 0)
237 };
238 } else {
239   return {
240     nextName: 'Diet Legend Unlocked 🏆',
241     targetPoints: 330,
242     pct: 100,
243     pointsLeft: 0
244 };
245 }
246 };
247
248 const milestone = getMilestoneDetails();
249
250 // Get specific challenge icon
251 const getChallengeIcon = (title) => {
252   const t = title.toLowerCase();
253   if (t.includes('water') || t.includes('hydration')) return Droplet;
254   if (t.includes('steps') || t.includes('walk')) return Footprints;
255   if (t.includes('sleep')) return Moon;
256   if (t.includes('meditate') || t.includes('mindful')) return Smile;
257   if (t.includes('post') || t.includes('community')) return MessageSquare;
258   return Target;
259 };
260
261 // Count completed badges/challenges today
262 const completedBadgesCount = challenges.filter(c => c.completed).length;
263
264 if (loading) {
265   return (
266     <div className="flex flex-col items-center justify-center min-h-[500px] gap-3">
267       <div className="animate-spin rounded-full h-12 w-12 border-4 border-emerald-500 border-t-transparent" />
268       <p className="text-sm font-sans text-gray-500">Loading your challenges...</p>
269     </div>
270   );
271 }
272
273 return (
274   <div className="max-w-6xl mx-auto space-y-8 pb-16">
275     {/* Confetti / Alert Toast */}
276     <showConfetti on={
277       <div className="fixed top-6 left-1/2 -translate-x-1/2 z-50 bg-emerald-500 text-white px-6 py-3.5 rounded-2xl shadow-2xl font-black text-sm animate-bounce flex items-center gap-2 border-2 border-white">
278         <Sparkles size={18} />
279         {unlockedMessage}
280       </div>
281     } />
282
283     {/* Header Banner */}
284     <div className="flex flex-col md:flex-row md:items-center justify-between gap-6 bg-white border border-gray-150 rounded-1xl p-6 sm:p-8 shadow-sm">
285       <div className="space-y-2">
286         <div className="flex items-center gap-3">
287           <p className="p-3 bg-emerald-50 text-emerald-700 rounded-2xl">
288             <Copy size={28} />
289           </p>
290           <div>
291             <div className="text-2xl sm:text-3xl font-extrabold text-gray-500 tracking-tight">
292               Daily Habit Challenges
293             </div>
294             <div className="text-sm text-gray-500 font-medium sm:font-medium">
295               Complete daily micro-habits, log custom amounts, earn points & collect tier milestones.
296             </div>
297           </div>
298         </div>
299       </div>
300
301       {/* Global Streak & Points Pill */}
302       <div className="flex flex-wrap items-center gap-3">
303         <div className="flex items-center gap-2 bg-amber-50 border border-amber-200 px-4 py-2.5 rounded-2xl">
304           <div className="text-amber-500 fill-amber-500" size={28} />

```

```

client > src > pages > Challenges > Challenges.jsx > ...
84 export default function Challenges() {
85   <div>
86     <span className="block text-[18px] font-bold text-amber-700 uppercase">Streak</span>
87     <span className="text-sm font-black text-gray-900">{currentStreak} Days</span>
88   </div>
89   </div>
90
91   <div className="flex items-center gap-2 bg-emerald-50 border border-emerald-200 px-4 py-2.5 rounded-2xl">
92     <Sparkles className="text-emerald-600" size={20} />
93     <div>
94       <span className="block text-[18px] font-bold text-emerald-700 uppercase">Today's Points</span>
95       <span className="text-sm font-black text-gray-900">{todayPoints} / 330 pts</span>
96     </div>
97   </div>
98   </div>
99 </div>
100
101 /* Milestone Level Progress Section */
102 <div className="bg-gradient-to-br from-emerald-600 to-teal-700 rounded-3xl p-6 sm:p-8 text-white shadow-xl space-y-6 relative overflow-hidden">
103   <div className="relative z-10 flex flex-col md:flex-row md:items-center justify-between gap-4">
104     <div>
105       <span className="text-[18px] font-extrabold uppercase tracking-wider bg-white/20 px-3 py-1 rounded-lg">
106         Next Tier Milestone
107       </span>
108       <h2 className="text-xl sm:text-2xl font-black mt-2">
109         Level Target: {milestone.nextName}
110       </h2>
111       <p className="text-xs text-emerald-100 font-medium mt-1">
112         {milestone.pointsLeft} > 0
113         ? 'Earn ${milestone.pointsLeft} more points to unlock this achievement.'
114         : 'All primary milestone levels unlocked! Keep challenging yourself!'
115       </p>
116     </div>
117
118     <div className="bg-white/10 backdrop-blur-md px-5 py-3 rounded-2xl border border-white/20 text-center">
119       <span className="text-[18px] font-extrabold uppercase text-emerald-200 block">Current Level</span>
120       <span className="text-lg font-black">
121         {userPoints} >= 330
122         ? 'Diet Legend 🏆'
123         : userPoints >= 200
124         ? 'Nutrition Master'
125         : userPoints >= 100
126         ? 'Healthy Starter'
127         : 'Beginner'
128       </span>
129     </div>
130   </div>
131
132   /* Milestone Progress bar */
133   <div className="relative z-10 space-y-2">
134     <div className="flex justify-between text-xs font-bold text-emerald-100">
135       <span>{userPoints} pts</span>
136       <span>{milestone.targetPoints} pts</span>
137     </div>
138     <div className="w-full bg-black/20 rounded-full h-3.5 p-0.5 overflow-hidden border border-white/10">
139       <div
140         className="bg-white h-full rounded-full transition-all duration-700 shadow-sm"
141         style={{ width: `${milestone pct}%` }}
142       </div>
143     </div>
144   </div>
145 </div>

```

```

client > src > pages > Challenges > Challenges.jsx > ...
84   export default function Challenges() {
362       />
363     </div>
364   </div>
365 </div>
366
367   /* Daily Challenges Section */
368   <div className="space-y-4">
369     <div className="flex items-center justify-between px-2">
370       <div>
371         <h2 className="text-xl font-bold text-gray-900">Today's Challenges</h2>
372         <p className="text-xs text-gray-500 font-semibold mt-0.5">
373           Completed {completedBadgesCount} of {challenges.length} challenges today
374         </p>
375       </div>
376     </div>
377
378     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
379       {challenges.map((c) => {
380         const Icon = getChallengeIcon(c.title);
381         const isCompleted = c.completed;
382         const progressPct = Math.min(Math.round((c.current / c.target) * 100), 100);
383         const loadingAction = actionLoading[c._id];
384
385         return (
386           <div
387             key={c._id}
388             className={`bg-white border rounded-3xl p-6 shadow-sm flex flex-col justify-between transition-all duration-300 ${
389               isCompleted
390                 ? 'border-emerald-200 bg-emerald-50/20'
391                 : 'border-gray-150 hover:border-emerald-300 hover:shadow-md'
392             }`}
393           >
394             <div className="space-y-4">
395               /* Top Row: Icon & Points Reward */
396               <div className="flex items-center justify-between">
397                 <span
398                   className={`p-3 rounded-2xl ${
399                     isCompleted
400                       ? 'bg-emerald-100 text-emerald-700'
401                       : 'bg-gray-100 text-gray-700'
402                   }`}
403                 >
404                   <Icon size={20} />
405                 </span>
406                 <span className="text-xs font-black text-amber-600 bg-amber-50 px-2.5 py-1 rounded-lg border border-amber-200">
407                   {c.pointsReward} pts
408                 </span>
409               </div>
410
411               /* Title & Description */
412               <div>
413                 <h3 className="text-base font-extrabold text-gray-900 leading-snug">{c.title}</h3>
414                 <p className="text-xs text-gray-500 mt-1 leading-relaxed">{c.description}</p>
415               </div>
416
417               /* Increment Progress Bar */

```



```

524 export default function Challenges() {
525   (badgesConfig.map((badge) => {
526     const isMilestoneEarned =
527       (badge.key === 'diet_legend' && todayPoints >= 330) ||
528       (badge.key === 'nutrition_master' && todayPoints >= 200) ||
529       (badge.key === 'healthy_starter' && todayPoints >= 100);
530
531     const isUnlocked = isHabitCompleted || isMilestoneEarned;
532     const IconComponent = badge.icon;
533
534     return (
535       <div
536         key={badge.key}
537         className={`flex flex-col items-center text-center p-5 rounded-3xl border transition-all duration-300 relative overflow-hidden group ${
538           isUnlocked
539             ? 'bg-white border-emerald-200 shadow-md shadow-emerald-500/5'
540             : 'bg-gray-50/50 border-gray-200/60 opacity-50'
541         }`}
542       >
543         <div>
544           <div>
545             <div>
546               <div>
547                 <div>
548                   <div>
549                     <div>
550                       <div>
551                         <div>
552                           <div>
553                             <div>
554                               <div>
555                                 <div>
556                                   <div>
557                                     <div>
558                                       <div>
559                                         <div>
560                                           <div>
561                                             <div>
562                                               <div>
563                                                 <div>
564                                                   <div>
565                                                     <div>
566                                                       <div>
567                                                         <div>
568                                                           <div>
569                                                             <div>
570                                                               <div>
571                                                                 <div>
572                                                                  <div>
573                                                                   <div>
574                                                                    <div>
575                                                                     <div>
576                                                                      <div>
577                                                                       <div>
578                                                                        <div>
579                                                                         <div>
580                                                                          <div>
581                                                                           <div>
582                                                                            <div>

```

AI Image Calorie Counter Feature

The AI Calorie Estimator is accessible via the "Scan Meal" tab (activeTab === 'image'), which is the default active tab when the user lands on the /meal-log page. On desktop, the UI renders a drag-and-drop zone where users can drag a photo or click to open a file browser (fileInputRef), accepting JPEG/PNG up to 10MB. On mobile, it switches to two dedicated buttons, which are, Take Photo (opens device camera via mobileCameraRef) and Photo Library (opens gallery via mobileGalleryRef), then detected automatically through navigator.userAgent. Once an image is selected via handleImageSelect(), a preview is shown. The user then clicks "Analyse with Gemini AI", which triggers handleAnalyseImage(), this builds a FormData object with the image, meal type, and date, then calls logMealImage() from mealLogService.js (a multipart/form-data POST to /meal-logs/image). The result is displayed in the <ResultCard> component showing detected food name, calorie count, macro bars (carbs, protein, fat), confidence badge, and item breakdown table. Free users are limited to 2 scans/day, with a Pro upgrade prompt on limit hit.


```

4 import React, { useState, useEffect, useRef, useCallback, useMemo } from 'react';
5 import { Link } from 'react-router-dom';
6 import { logMealText, logMealImage, getDailyLog, deleteMealLog } from '../services/mealLogService';
7 import { useAuth } from '../context/AuthContext';
8 import {
9   Camera,
10   Pencilline,
11   Sunrise,
12   Sun,
13   Moon,
14   Apple,
15   Utensils,
16   Crown,
17   Sparkles,
18   X,
19   CheckCircle,
20   CheckCircle2,
21   Minus,
22   HelpCircle,
23   RefreshCw,
24   ImageUp,
25   AlertTriangle,
26   ListChecks,
27   Images,
28   CalendarDays,
29   CalendarRange,
30   ChevronDown,
31 } from 'lucide-react';

```

```

client > src > pages > MealLog > JS MealLog.jsx > ...
32
33 // --- Helper: today as YYYY-MM-DD (Local timezone) ---
34 const todayString = () => {
35   const d = new Date();
36   const year = d.getFullYear();
37   const month = String(d.getMonth() + 1).padStart(2, '0');
38   const day = String(d.getDate()).padStart(2, '0');
39   return `${year}-${month}-${day}`;
40 };
41
42 // --- Confidence badge component ---
43 function ConfidenceBadge({ level }) {
44   const colours = {
45     high: 'bg-emerald-50 text-emerald-700 border-emerald-200',
46     medium: 'bg-amber-50 text-amber-700 border-amber-200',
47     low: 'bg-rose-50 text-rose-700 border-rose-200',
48   };
49   const IconMap = { high: CheckCircle2, medium: Minus, low: HelpCircle };
50   const Icon = IconMap[level] || Minus;
51   return (
52     <span className={`inline-flex items-center gap-1 text-xs font-semibold px-2.5 py-0.5 rounded-full border ${colours[level]} || colours.medium`} >
53       <Icon size={11} />
54       {level ? level.charAt(0).toUpperCase() + level.slice(1) : 'Medium'} Confidence
55     </span>
56   );
57 }
58
59 // --- Macro bar component ---
60 function MacroBar({ label, value, unit, colour, max }) {
61   const pct = max > 0 ? Math.min(Math.round((value / max) * 100), 100) : 0;
62   return (
63     <div>
64       <div className="flex justify-between text-xs font-medium mb-1.5">
65         <span className="text-gray-500">{label}</span>
66         <span className="text-gray-900 font-semibold">{value}{unit}</span>
67       </div>
68       <div className="h-2 bg-gray-100 rounded-full overflow-hidden">
69         <div className={`h-full rounded-full transition-all duration-700 ${colour}`} style={{ width: `${pct}%` }} />
70       </div>
71     </div>
72   );
73 }
74
75 // --- Results card shown after Gemini analysis ---
76 function ResultCard({ result, onSave, saving, saved }) {
77   if (!result) return null;
78   return (
79     <div className="mt-6 bg-gray-50 border border-gray-200 rounded-2xl p-5 space-y-5 animate-[fadeIn_0.4s_ease]">
80       <div className="flex items-start justify-between gap-3 flex-wrap">
81         <div>
82           <p className="text-[11px] uppercase tracking-wider text-gray-500 mb-0.5">Detected</p>
83           <h3 className="text-base font-bold text-gray-900 leading-snug">{result.foodName}</h3>
84         </div>
85       </div>

```

```

86 |     <ConfidenceBadge level={result.confidence} />
87 |   </div>
88 |
89 |   /* Big calorie number */
90 |   <div className="flex items-end gap-2">
91 |     <span className="text-5xl font-black text-gray-900 tabular-nums">{result.calories}</span>
92 |     <span className="text-gray-500 text-sm mb-2">kcal</span>
93 |   </div>
94 |
95 |   /* Macro bars */
96 |   <div className="space-y-3">
97 |     <MacroBar label="Carbohydrates" value={result.carbs} unit="g" colour="bg-amber-400" max={150} />
98 |     <MacroBar label="Protein" value={result.protein} unit="g" colour="bg-emerald-400" max={80} />
99 |     <MacroBar label="Fat" value={result.fat} unit="g" colour="bg-rose-400" max={80} />
100 |   </div>
101 |
102 |   /* Per-item breakdown */
103 |   {result.breakdown && result.breakdown.length > 0 && (
104 |     <div>
105 |       <p className="text-[11px] uppercase tracking-wider text-gray-500 mb-2">Breakdown</p>
106 |       <div className="rounded-xl overflow-hidden border border-gray-200">
107 |         <table className="w-full text-xs">
108 |           <thead>
109 |             <tr className="bg-gray-50 text-gray-500">
110 |               <th className="text-left px-3 py-2 font-medium">Item</th>
111 |               <th className="text-left px-3 py-2 font-medium">Portion</th>
112 |               <th className="text-right px-3 py-2 font-medium">kcal</th>
113 |             </tr>
114 |           </thead>
115 |           <tbody>
116 |             {result.breakdown.map((b, i) => (
117 |               <tr key={i} className="border-t border-gray-200 hover:bg-gray-50 transition-colors">
118 |                 <td className="px-3 py-2 text-gray-700">{b.item}</td>
119 |                 <td className="px-3 py-2 text-gray-500">{b.portionEstimate}</td>
120 |                 <td className="px-3 py-2 text-right text-gray-900 font-semibold">{b.calories}</td>
121 |               </tr>
122 |             ))}
123 |           </tbody>
124 |         </table>
125 |       </div>
126 |     </div>
127 |   )}
128 |
129 |   /* Save button */
130 |   {!saved ? (
131 |     <button
132 |       onClick={onSave}
133 |       disabled={!saving}
134 |       className="w-full py-3 rounded-xl font-semibold text-sm bg-emerald-600 hover:bg-emerald-500 disabled:opacity-50 disabled:cursor-not-allowed text-white transition shadow-lg shadow-e
135 |     >
136 |       {saving ? (
137 |         <span className="animate-spin h-4 w-4 border-2 border-white/30 border-t-white rounded-full" />Saving...</>
138 |       ) : 'Save to Log'}
139 |     </button>
140 |   ) : (
141 |     <div className="w-full py-3 rounded-xl font-semibold text-sm bg-emerald-50 border border-emerald-200 text-emerald-700 text-center flex items-center justify-center gap-2">
142 |       <CheckCircle size={16} /> Saved to your meal log
143 |     </div>
144 |   )}
145 | </div>
146 | );
147 | }
148 |

```

```

149 // — Main page —
150 export default function MealLog() {
151   const { user } = useAuth();
152
153   // — Tab state —
154   const [activeTab, setActiveTab] = useState('image'); // 'image' | 'text'
155   const [selectedMealDate, setSelectedMealDate] = useState(() => {
156     try {
157       return new URLSearchParams(window.location.search).get('date') || todayString();
158     } catch {
159       return todayString();
160     }
161   });
162
163   // — Image upload state —
164   const [imageFile, setImageFile] = useState(null);
165   const [imagePreview, setImagePreview] = useState(null);
166   const [imageMealType, setImageMealType] = useState('lunch');
167   const [imageDragging, setImageDragging] = useState(false);
168   const [imageAnalysing, setImageAnalysing] = useState(false);
169   const [imageResult, setImageResult] = useState(null);
170   const [imageError, setImageError] = useState('');
171   const [imageSaving, setImageSaving] = useState(false);
172   const [imageSaved, setImageSaved] = useState(false);
173   const fileInputRef = useRef(null); // desktop: generic file browse
174   const mobileCameraRef = useRef(null); // mobile: take photo with camera
175   const mobileGalleryRef = useRef(null); // mobile: pick from photo library
176
177   // — Mobile detection —
178   const [isMobile, setIsMobile] = useState(false);
179   useEffect(() => {
180     const checkMobile = () =>
181       setIsMobile(window.matchMedia('(max-width: 767px) and (pointer: coarse)').matches
182         || /Android|iPhone|iPad|iPod|Mobile/i.test(navigator.userAgent));
183     checkMobile();
184     window.addEventListener('resize', checkMobile);
185     return () => window.removeEventListener('resize', checkMobile);
186   }, []);
187
188   // — Text log state —
189   const [textFood, setTextFood] = useState('');
190   const [textPortion, setTextPortion] = useState('');
191   const [textMealType, setTextMealType] = useState('lunch');
192   const [textAnalysing, setTextAnalysing] = useState(false);
193   const [textResult, setTextResult] = useState(null);
194   const [textError, setTextError] = useState('');
195   const [textSaving, setTextSaving] = useState(false);
196   const [textSaved, setTextSaved] = useState(false);
197
198   // — History state —
199   const [historyPeriod, setHistoryPeriod] = useState('daily'); // 'daily' | 'weekly' | 'monthly'
200   const [historyDate, setHistoryDate] = useState(todayString()); // for daily
201   const [historyWeek, setHistoryWeek] = useState(todayString()); // any day in the desired week
202   const [historyMonth, setHistoryMonth] = useState(todayString().slice(0, 7)); // 'YYYY-MM'
203   const [historyLogs, setHistoryLogs] = useState([]);
204   const [historyTotals, setHistoryTotals] = useState({ calories: 0, carbs: 0, protein: 0, fat: 0 });
205   const [historyLoading, setHistoryLoading] = useState(false);
206
207   // — Scans Left state —
208   const [scansLeft, setScansLeft] = useState(2);

```

```

306 // — Image handlers —
307 const handleImageSelect = (file) => {
308   if (!file || !file.type.startsWith('image/')) {
309     setImageError('Please select a valid image file (JPEG or PNG).');
310     return;
311   }
312   setImageFile(file);
313   setImageError('');
314   setImageResult(null);
315   setImageSaved(false);
316   const reader = new FileReader();
317   reader.onloadend = () => setImagePreview(reader.result);
318   reader.readAsDataURL(file);
319 };
320
321 const handleDrop = (e) => {
322   e.preventDefault();
323   setImageDragging(false);
324   const file = e.dataTransfer.files[0];
325   handleImageSelect(file);
326 };
327
328 const handleAnalyseImage = async () => {
329   if (!imageFile) { setImageError('Please select a meal image first.');
```

```

150 export default function MealLog() {
151   /* -- IMAGE TAB -- */
152   {activeTab === 'image' && (
153     <div className="space-y-5">
154       /* Meal type selector */
155       <div className="flex flex-wrap gap-2">
156         {MEAL_TYPES.map(t => {
157           const MealIcon = MEAL_ICON_MAP[t] || Utensils;
158           return (
159             <button
160               key={t}
161               onClick={() => setImageMealType(t)}
162               className={`inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl text-xs font-semibold capitalize transition ${
163                 imageMealType === t
164                   ? `bg-emerald-100 border border-emerald-300 text-emerald-700`
165                   : `bg-gray-50 border border-gray-200 text-gray-500 hover:text-gray-900 hover:bg-gray-100`
166               }`}
167             <MealIcon size={13} />
168             {t}
169           )
170         })}
171       </div>
172     )
173   )
174   /* -- Upload area: desktop drag-drop OR mobile picker buttons -- */
175   {isMobile ? (
176     /* -- MOBILE: three action buttons -- */
177     <div className="space-y-3">
178       /* Preview (shown after any selection) */
179       {imagePreview && (
180         <div className="relative rounded-2xl overflow-hidden border border-gray-200">
181           <img
182             src={imagePreview}
183             alt="Meal preview"
184             className="w-full max-h-64 object-contain bg-gray-50"
185           />
186           <button
187             onClick={() => { setImageFile(null); setImagePreview(null); setImageResult(null); setImageSaved(false); setImageError(''); }}
188             className="absolute top-2 right-2 p-1.5 bg-white/90 hover:bg-white text-gray-500 hover:text-rose-500 rounded-xl border border-gray-200 shadow-sm transition"
189           />
190           <X size={14} />
191         </div>
192       )
193     )
194     /* Two picker buttons */
195     {!user?.isPro && (
196       <div className="mb-2 text-center">
197         <span className="inline-block px-3 py-1 bg-emerald-50 text-emerald-700 text-xs font-semibold rounded-full border border-emerald-200">
198           {scansLeft} scans left today
199         </span>
200       </div>
201     )
202   )
203 }

```

```

150 export default function Mealog() {
151
152   /* Take Photo - opens device camera */
153   <button
154     onClick={() => mobileCameraRef.current?.click()}
155     className="flex flex-col items-center gap-2 py-5 rounded-2xl border-2 border-dashed border-emerald-300 border-emerald-50 hover:bg-emerald-100 text-emerald-700 transition active:scale-95"
156   >
157     <Camera size={24} />
158     <span className="text-[11px] font-semibold text-center leading-tight">Take<br/>Photo</span>
159   </button>
160
161   /* Photo Library - opens gallery */
162   <button
163     onClick={() => mobileGalleryRef.current?.click()}
164     className="flex flex-col items-center gap-2 py-5 rounded-2xl border-2 border-dashed border-teal-300 border-teal-50 hover:bg-teal-100 text-teal-700 transition active:scale-95"
165   >
166     <Images size={24} />
167     <span className="text-[11px] font-semibold text-center leading-tight">Photo<br/>Library</span>
168   </button>
169 </div>
170
171 /* Hidden inputs for each mobile source */
172 /* Camera capture */
173 <input
174   ref={mobileCameraRef}
175   type="file"
176   accept="image/*"
177   capture="environment"
178   className="hidden"
179   onChange={(e) => handleImageSelect(e.target.files[0])}
180 />
181
182 /* Photo library */
183 <input
184   ref={mobileGalleryRef}
185   type="file"
186   accept="image/*"
187   className="hidden"
188   onChange={(e) => handleImageSelect(e.target.files[0])}
189 />
190 </div>
191 ) : (
192   /* --- DESKTOP: drag & drop zone --- */
193   <>
194     <div
195       onDragOver={(e) => { e.preventDefault(); setImageDragging(true); }}
196       onDragLeave={() => setImageDragging(false)}
197       onDrop={handleDrop}
198       onClick={() => fileInputRef.current?.click()}
199       className="relative cursor-pointer rounded-2xl border-2 border-dashed transition-all flex flex-col items-center justify-center min-h-[200px] ${
200         imageDragging
201           ? `border-emerald-500 border-emerald-50`
202           : `border-gray-200 bg-transparent`
203         }
204         border-gray-200 hover:border-emerald-500 border-gray-50 hover:bg-gray-100/30"
205     >
206       <ImagePreview
207         : `border-gray-200 bg-transparent`
208       </ImagePreview>
209     </div>
210   </>
211 )
212 }

```

```

150 export default function MealLog() {
151   |   </Link>
152   |   </div>
153   </div>
154   ): <ImageError ss (
155     <div className="flex items-center gap-2" <div text-rose-600 text-sm mt-2 <div bg-rose-50 border <div border-rose-200 px-3 py-2 rounded-xl">
156       <AlertTriangle size={15} className="shrink-0" />
157       <ImageError
158     </div>
159   )}
160
161   {/* Analyse button */}
162   <button
163     |   onClick={handleAnalyseImage}
164     |   disabled={!imageFile || !imageAnalysing}
165     |   className="w-full py-3.5 rounded-xl font-bold text-sm bg-gradient-to-r from-emerald-600 to-teal-600 hover:from-emerald-500 hover:to-teal-500 disabled:opacity-40 disabled:cursor-not-allowed text-white transition
166   >
167     <div>
168       <div>
169         <RefreshCw size={16} className="animate-spin" />
170         Gemini is analysing your meal.
171       </div>
172     </div>
173   </button>
174
175   {/* Results */}
176   <ResultCard
177     |   results={imageResult}
178     |   saved={imageSaved}
179     |   saving={imageSaving}
180     |   onSave={() => {}} // auto-saved on analysis
181   />
182 </div>
183 }
184 }

```

AI Text Calorie Counter Feature

The AI Meal Log is accessible via the "Log by Text" tab (activeTab === 'text'), toggled from the same tab bar at the top of the log panel on the /meal-log page. Users select a meal type (Breakfast, Lunch, Dinner, Snacks) via pill buttons, then fill in a Food Name input field (e.g. "Dal Bhat, Biryani") and an optional Portion Size field. Pressing Enter or clicking "Estimate with Gemini AI" triggers `handleAnalyseText()`, which calls `logMealText()` from `mealLogService.js` (a JSON POST to /meal-logs) with the food name, portion description, meal type, and selected date. The AI result is rendered identically in the shared `<ResultCard>` component, showing food name, calorie count, macro bars, and a confidence badge. Both tabs share the Meal History section below the log panel, which automatically refreshes after every successful analysis and supports daily, weekly, and monthly filtering via a dropdown, allowing users to review all past AI-logged meals in one place.

```
356 // — Text handlers —
357 const handleAnalyseText = async () => {
358   if (!textFood.trim()) { setTextError('Please enter a food name.');
```

`return; }`

```
359   setTextAnalysing(true);
360   setTextError('');
361   setResult(null);
362   setTextSaved(false);
363   try {
364     const { data } = await logMealText({
365       foodName: textFood.trim(),
366       portionDescription: textPortion.trim(),
367       mealType: textMealType,
368       date: selectedMealDate,
369     });
370     setResult(data.log);
371     setTextSaved(true); // auto-saved on analysis
372     fetchHistory(historyAnchor, historyPeriod);
373   } catch (err) {
374     setTextError(err.response?.data?.message || 'Failed to estimate nutrition. Please try again.');
```

`finally {`

```
375     setTextAnalysing(false);
376   }
377 }
378 };
```

```

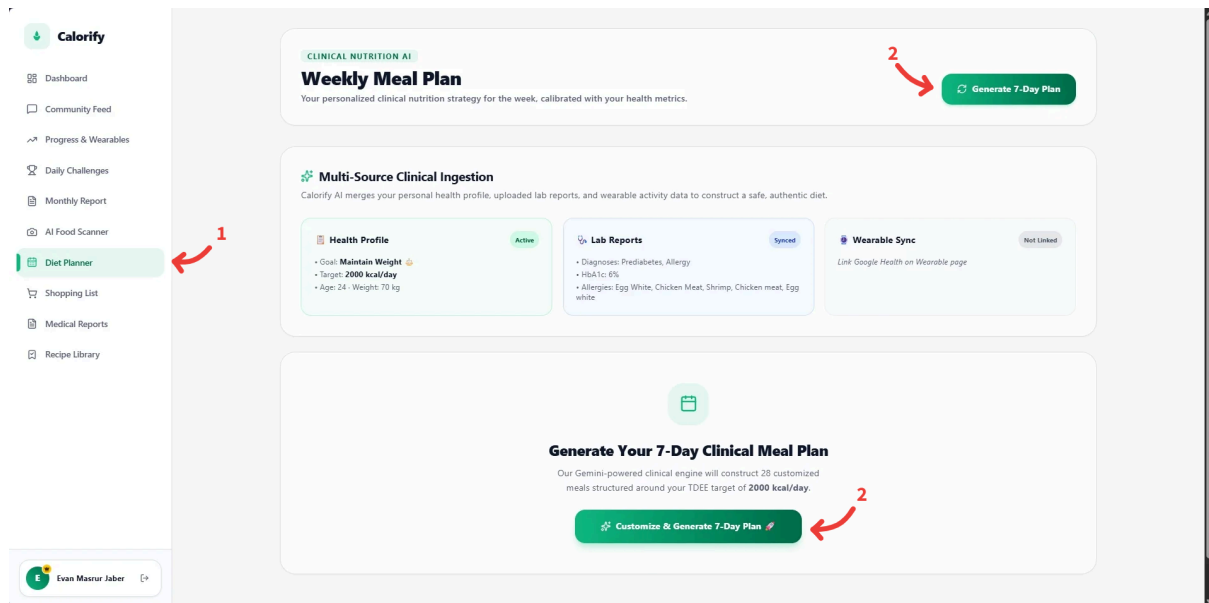
630      /* -- TEXT TAB -- */
631      {activeTab === 'text' && (
632        <div className="space-y-5">
633          /* Meal type selector */
634          <div className="flex flex-wrap gap-2">
635            {MEAL_TYPES.map(t => {
636              const MealIcon = MEAL_ICON_MAP[t] || Utensils;
637              return (
638                <button
639                  key={t}
640                  onClick={() => setTextMealType(t)}
641                  className={`inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl text-xs font-semibold capitalize transition ${
642                    textMealType === t
643                      ? 'bg-emerald-100 border border-emerald-300 text-emerald-700'
644                      : 'bg-gray-50 border border-gray-200 text-gray-500 text-gray-900 hover:bg-gray-100'
645                  }`}
646                >
647                  <MealIcon size={13} />
648                  {t}
649                </button>
650              );
651            })}
652          </div>
653
654          /* Food name */
655          <div>
656            <label className="block text-xs font-semibold text-gray-500 uppercase tracking-wider mb-1.5">Food Name</label>
657            <input
658              type="text"
659              value={textFood}
660              onChange={(e) => { setTextFood(e.target.value); setError(''); setResult(null); setSaved(false); }}
661              onKeyDown={(e) => e.key === 'Enter' && handleAnalyseText()}
662              placeholder="e.g. Dal Bhat, Biryani, Roti with Tarkari."
663              className="w-full bg-white border border-gray-200 focus:border-emerald-500 outline-none text-gray-900 placeholder-gray-400 text-sm px-4 py-3 rounded-xl transition"
664            />
665          </div>
666
667          /* Portion (optional) */
668          <div>
669            <label className="block text-xs font-semibold text-gray-500 uppercase tracking-wider mb-1.5">
670              Portion Size <span className="text-gray-400 normal-case font-normal">(optional)</span>
671            </label>
672            <input
673              type="text"
674              value={textPortion}
675              onChange={(e) => setTextPortion(e.target.value)}
676              placeholder="e.g. 1 large plate, 2 pieces, 300g."
677              className="w-full bg-white border border-gray-200 focus:border-emerald-500 outline-none text-gray-900 placeholder-gray-400 text-sm px-4 py-3 rounded-xl transition"
678            />
679          </div>
680
681          {textError && (
682            <div className="flex items-center gap-2 text-rose-600 text-sm bg-rose-50 border border-rose-200 px-3 py-2 rounded-xl">
683              <AlertTriangle size={15} className="shrink-0" />
684              {textError}
685            </div>
686          )}
687
688          /* Estimate button */
689          <button
690            onClick={handleAnalyseText}
691            disabled={!textFood.trim() || textAnalysing}
692            className="w-full py-3.5 rounded-xl font-bold text-sm bg-gradient-to-r from-emerald-600 to-teal-600 hover:from-emerald-500 hover:to-teal-500 disabled:opacity-40 disabled:cursor-not-allowed text-white transition"
693          >
694            {textAnalysing ? (
695              <>
696                <RefreshCw size={16} className="animate-spin" />
697                Gemini is estimating nutrition.
698              </>
699            ) : (
700              <>
701                <Sparkles size={16} />
702                Estimate with Gemini AI
703              </>
704            )}
705          </button>
706
707          /* Results */
708          <ResultCard
709            result={textResult}
710            saved={textSaved}
711            saving={textSaving}
712            onSave={() => {}} // auto-saved on analysis
713          />
714        </div>
715      )}
716    </div>

```

7. User Manual

Diet Planning

From the Diet Planner tab (1), review your synced health profile, lab reports, and wearable data used to establish your clinical baseline. When ready, click Generate 7-Day Plan (2) at the top or Customize & Generate 7-Day Plan (2) below to create an AI-tailored 28-meal schedule structured around your daily caloric and nutritional targets.



After clicking the diet generator button in the pop up in Customize Your 7 Day Meal Plan modal, select one or more preferred cuisine styles (1) or specify a custom cuisine (2). Next, choose relevant dietary preferences and health constraints (3), or enter custom dietary goals (4). Add any specific ingredient restrictions or requests in the optional notes field (5). When finished, click Generate 7-Day Plan (6) to create your personalized weekly schedule, or select Cancel (7) to discard your selections

FULL WEEK GENERATION

×

Customize Your 7-Day Meal Plan

Select cuisine styles and dietary preferences to guide Gemini AI in generating your schedule.

1. SELECT CUISINE STYLE(S)

Multiple choice

☒

BD

Bangladeshi / Bengali

Authentic local home dishes (Shak, Ilish, Daal, Bharta)

☐

🍜

Pan-Asian

Thai, Japanese, Chinese stir-fries, steamed fish, soups

☐

🍷

Continental / Western

Grilled proteins, baked fish, roasted veggies, salads

☐

🍝

Italian

Whole wheat pasta, marinara, grilled chicken, minestrone

☐

🥗

Mediterranean

Greek salads, hummus, grilled seafood, olive oil dressings

☐

🍲

Middle Eastern

Shish tawook, lentil shorba, baked kababs, spiced bowls

☐

🌮

Mexican / Latin

Burrito bowls, fajitas, black beans, guacamole, tacos

☐

🍛

Pan-Indian

Tandoori proteins, dal tadka, palak, idli, aromatic curries

Or specify custom cuisine (e.g. Japanese, Greek, Fusion, Turkish...)

2. DIETARY PREFERENCES & HEALTH CONSTRAINTS

Multiple choice

☐

🔥

Diabetic-Friendly (Low GI)

Complex carbs, high fiber, zero refined sugars

☐

🥬

Low-Carb / Keto

Minimized rice/flour; focuses on lean proteins & greens

☐

🍖

High-Protein

Maximized protein density for muscle repair & satiety

☐

🥬

Low-Oil / Heart-Healthy

Steamed, boiled, air-fried with minimal cooking fats

☐

🌱

Vegetarian / Plant-Based

100% plant proteins: paneer, lentils, seasonal veggies

☐

🌾

Gluten-Free

Naturally gluten-free grains, vegetables, and proteins

☐

⚡

Quick & Easy (<20 mins)

Fast preparation times with accessible ingredients

☐

⚖️

Balanced Macro

Balanced carbs, protein, and healthy fats

Or specify custom dietary goal (e.g. Dairy-Free, Low-Sodium, Intermittent Fasting...)

CUSTOM DIETARY NOTES / REQUESTS (OPTIONAL)

e.g., more fish curries, avoid mustard oil, include fresh seasonal fruits, less spicy dinner...

7

6

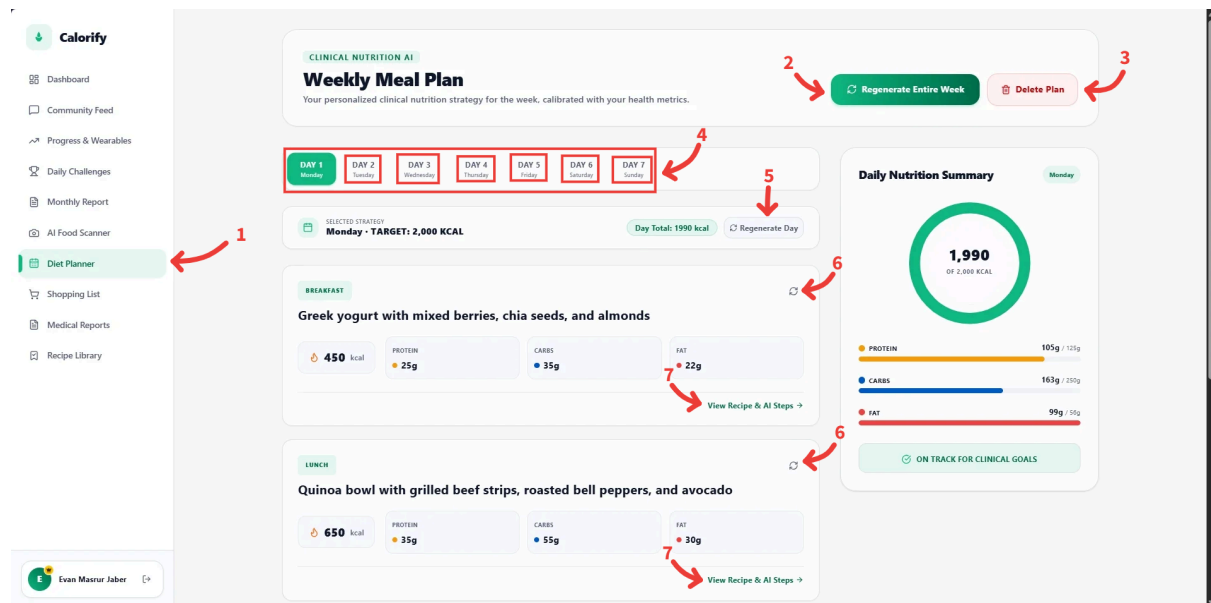
Cancel

Generate 7-Day Plan

From the Diet Planner tab (1), view and customize your AI calibrated weekly nutrition schedule. Switch between days (4) to inspect scheduled meals and track daily macronutrient targets on the right-hand summary card. Adjust your schedule at any level by regenerating individual meals (6), a single day (5), or the full week (2). Click View Recipe & AI Steps (7)

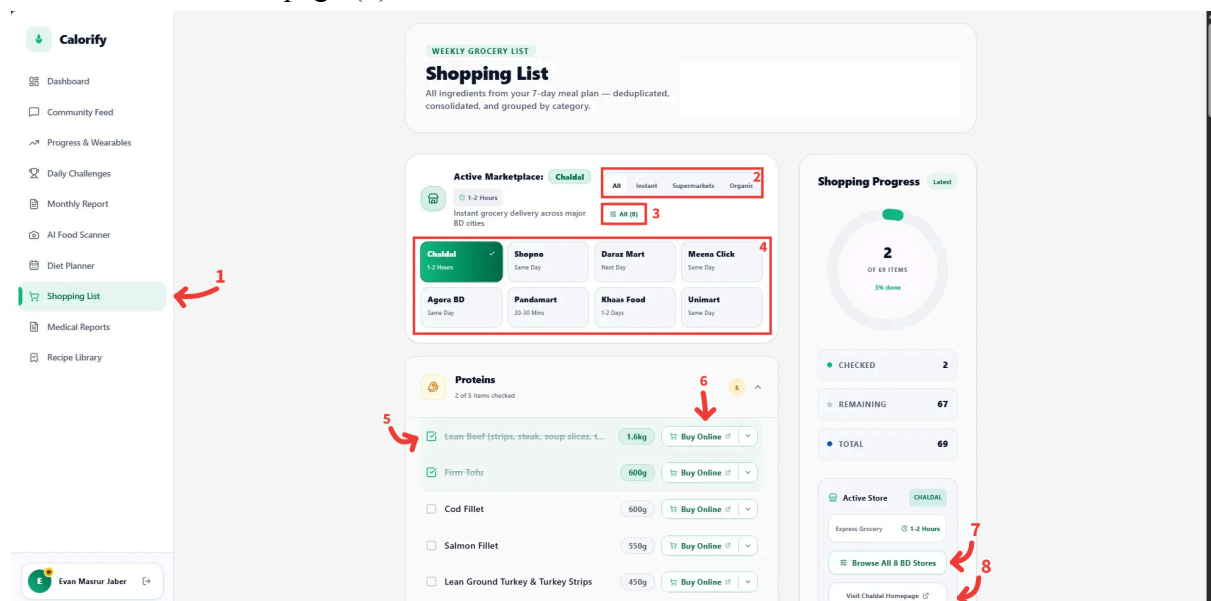
38

on any dish for step by step cooking instructions, or select Delete Plan (3) to clear the week and start fresh.



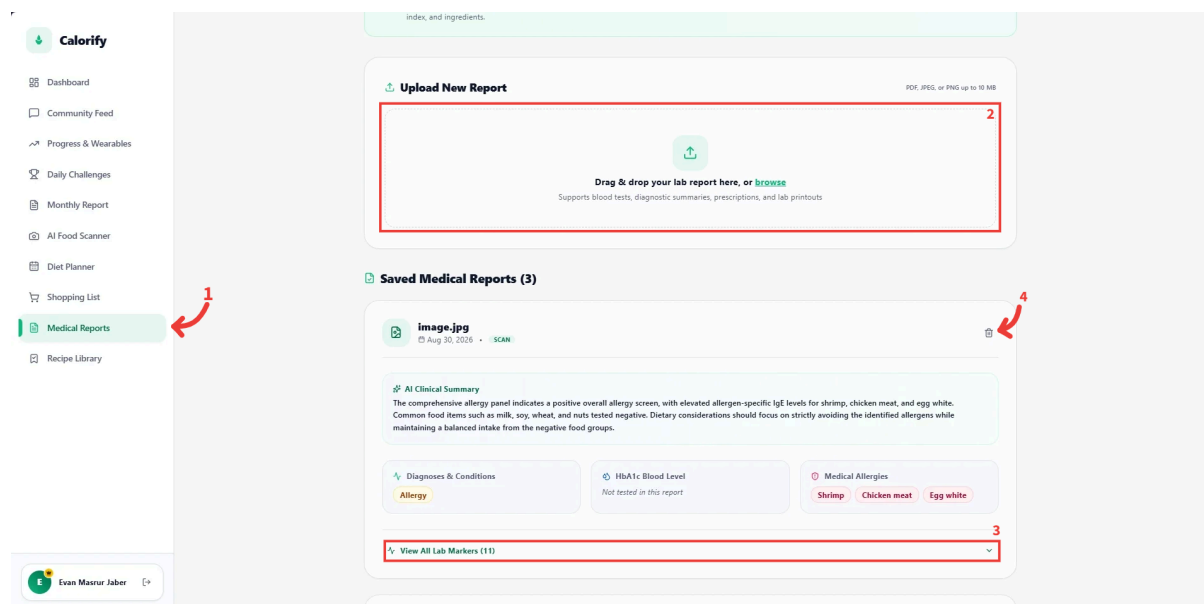
Shopping List Generator

From the Shopping List tab (1), view and manage your consolidated weekly ingredients grouped by category. Filter store types (2) or expand the directory view (3) to select your preferred active marketplace (4). Track your progress by ticking off items as you shop (5) or click Buy Online (6) to purchase individual ingredients directly. From the summary panel on the right, monitor your completion status, browse alternative local merchants (7), or open the active retailer's homepage (8).



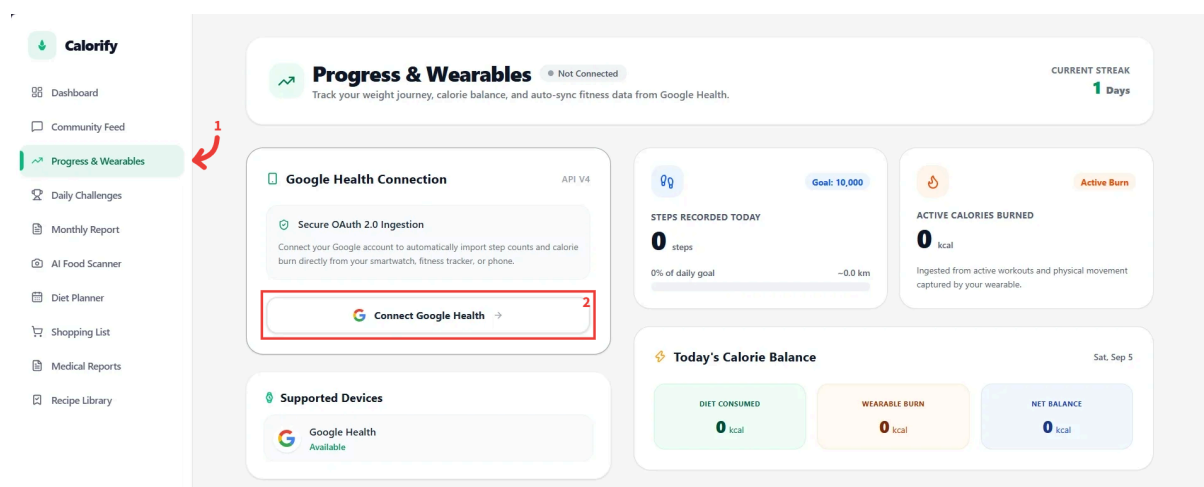
Medical Report Parsing

From the Medical Reports tab (1), upload diagnostic summaries, lab printouts, or prescriptions by dragging and dropping files or browsing your device (2). Below, review your saved reports to see AI-extracted clinical summaries, diagnosed conditions, and flagged allergies. Click View All Lab Markers (3) to expand and inspect specific biomarker levels, or select the trash icon (4) to remove a report from your records.

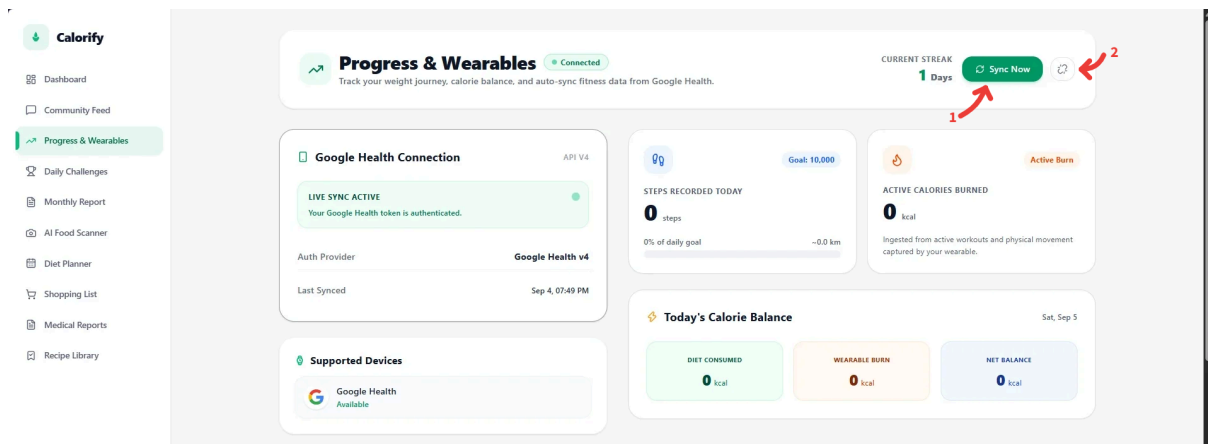


Wearable Data Ingestion

From the Progress & Wearables tab (1), track your daily activity, net calorie balance, and weight journey in one dashboard. Click Connect Google Health (2) to securely link your Google account via OAuth 2.0 and automatically import real time step counts and active calorie burn directly from your smartwatch, fitness tracker, or phone.

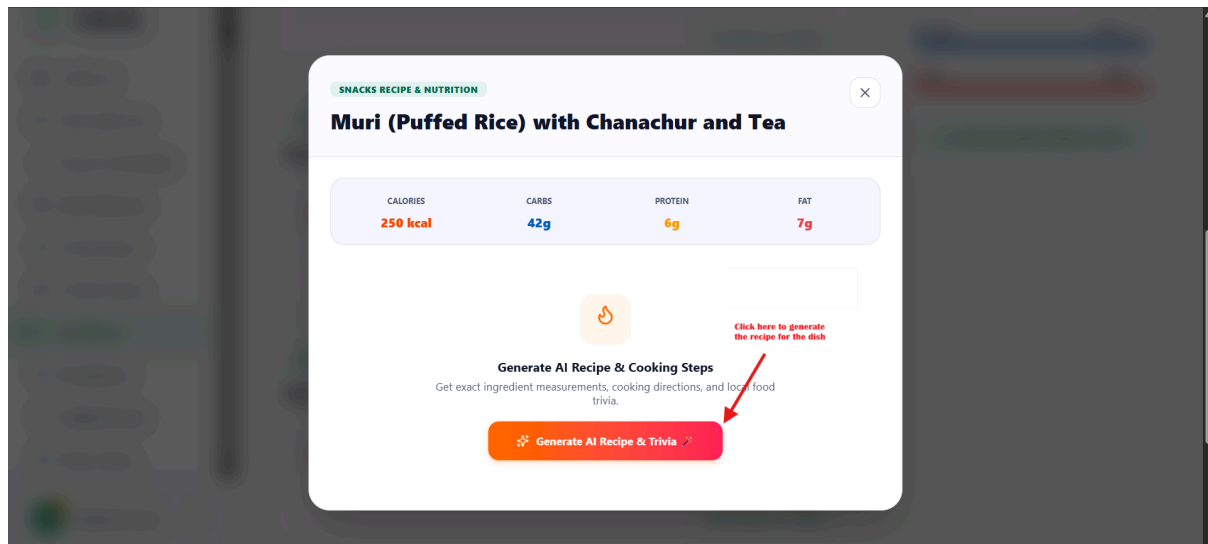


With your wearable account connected, click Sync Now (1) to manually pull the latest activity, step counts, and calorie burn data from Google Health. Click the settings icon (2) to manage your connection preferences, sync options, or unlink the integration.



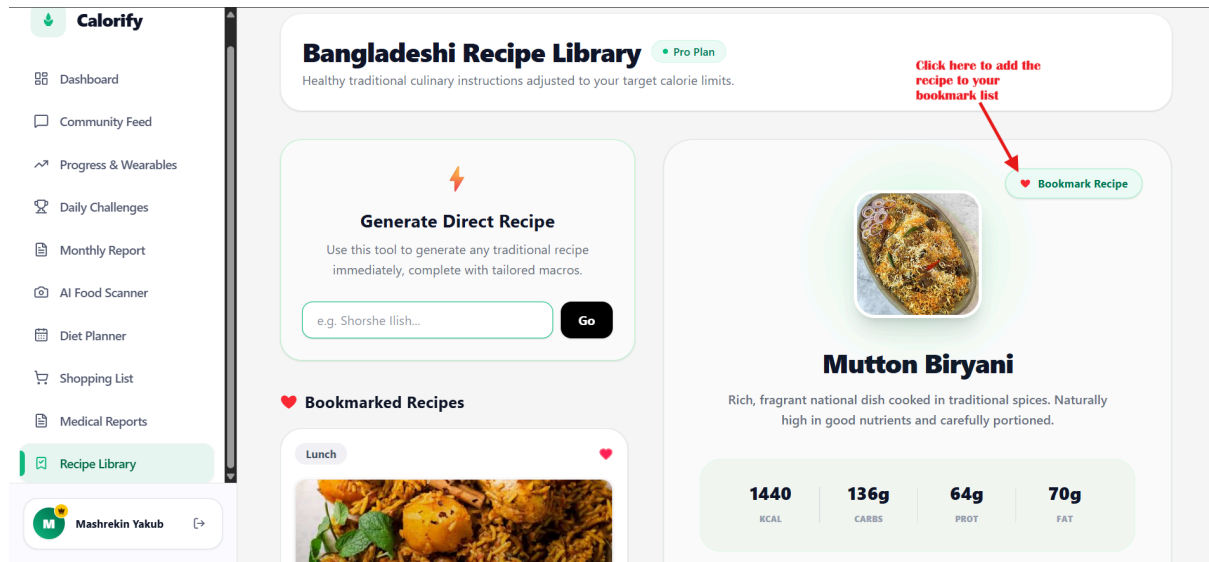
AI Recipe Generator

Every meal card in your diet plan or recipe library includes a nutrition summary (calories, carbs, protein, fat). To get the full cooking instructions for any dish, open its details popup and click Generate AI Recipe & Trivia. The system will generate exact ingredient measurements, step-by-step cooking directions, and a piece of local food trivia related to the dish, so you don't just see the macros, you get everything you need to actually cook it. Once generated, the recipe is saved and will load instantly the next time you open that same meal.



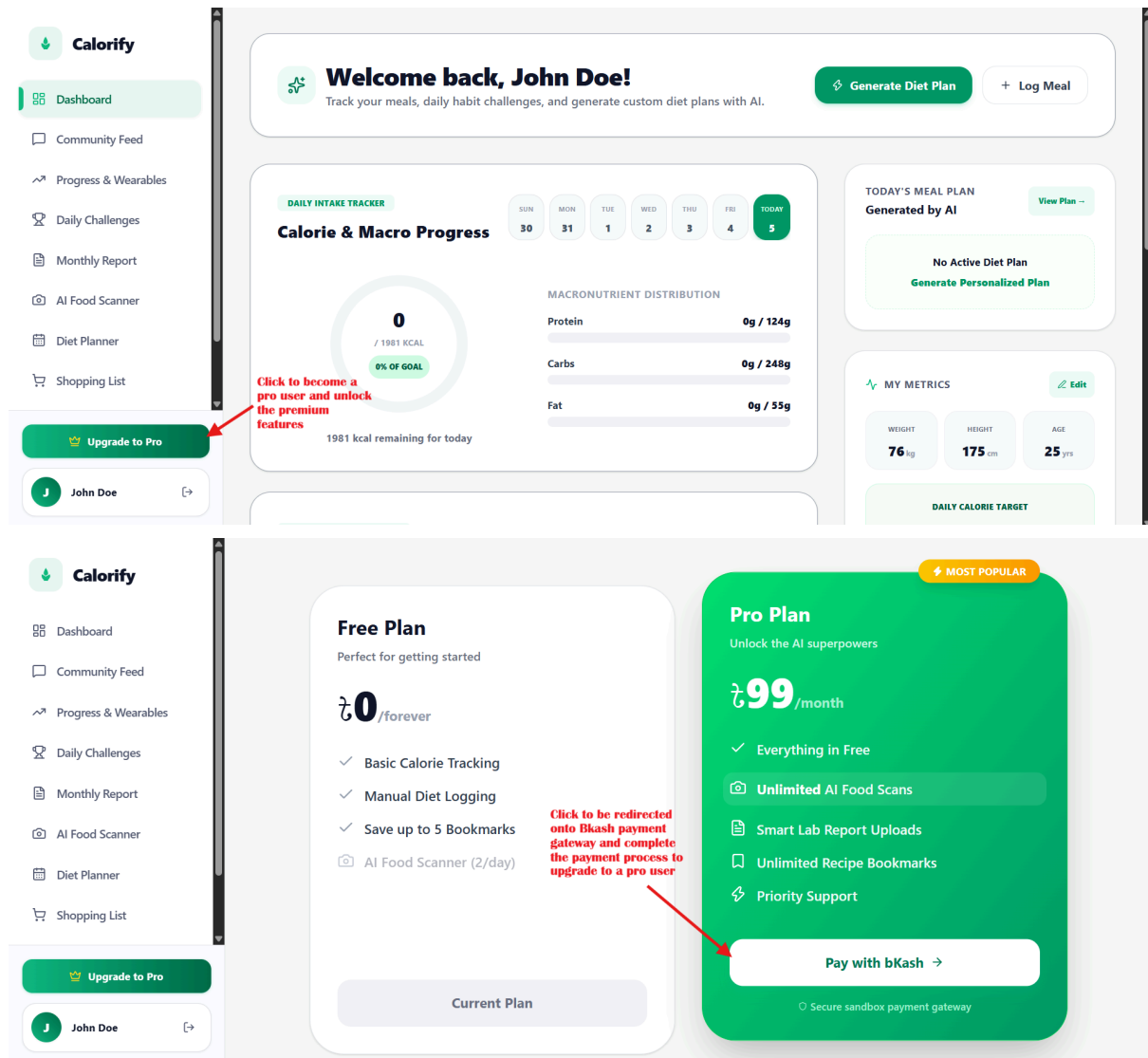
Recipe Bookmarking

Found a dish you want to come back to? Click the Bookmark Recipe button (heart icon) on any recipe in the Bangladeshi Recipe Library to add it to your personal saved collection, accessible anytime from the Bookmarked Recipes section. This is useful for building a rotating list of go-to meals instead of re-searching each week. Note that your plan tier affects this feature: Free Plan users can bookmark up to 5 recipes at a time, while Pro Plan users get unlimited bookmarks, the sidebar will show your remaining bookmark count if you're on the Free tier.



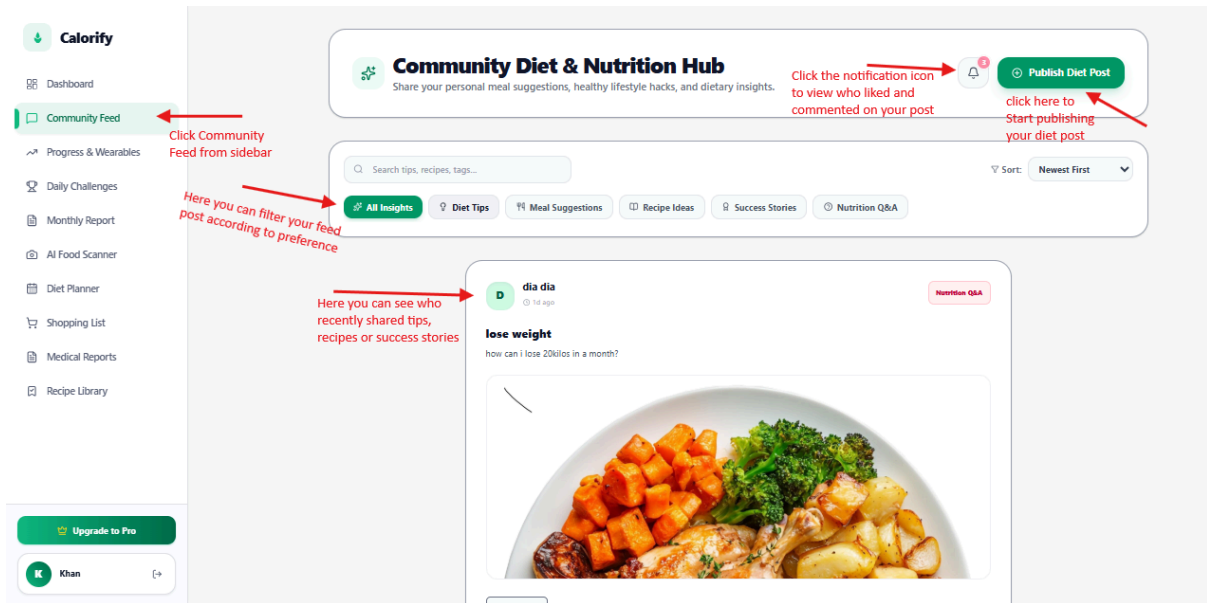
Pro Subscription & Payment Gateway

If you'd like to unlock premium features, click the Upgrade to Pro button in the sidebar (visible on the Dashboard and other pages) to open the subscription comparison page. Here you can see exactly what's included in the Free Plan versus the Pro Plan (₹99/month). Pro unlocks unlimited AI food scans, smart lab report uploads, unlimited recipe bookmarks, and priority support. When you're ready, click Pay with bKash to be securely redirected to bKash's checkout gateway. Once your payment is confirmed, your account is instantly upgraded and all Pro features unlock immediately, no need to log out or refresh.



Community Feed Feature

Click Community Feed in the sidebar to browse who recently shared tips, recipes, or success stories, and filter the posts by topic according to your preference. On any post, you can view the author's username, points, and post history, check the likes count, and read or add comments. Click the bell icon at the top to see notifications of who liked and commented on your posts, or click Publish Diet Post to create your own by choosing a category, entering a title, writing your thoughts or queries, uploading an optional photo, adding descriptive tags, and clicking Publish Post.



Zephrox 2d ago ★ 150 pts Meal Suggestion

Cheat day
1500+ calories in 1 sitting after 1 week of 400 calories deficit daily



♥ 9 💬 2 Meal Suggestion

Comments:

- Evan Masrur Jaber 2d ago
- dia dia 1d ago
not cheat day allowed!!

Add a suggestion or reply...

Annotations:
 - Red arrow points to 'Zephrox': You can view username, points and their post history
 - Red arrow points to '♥ 9': Likes count
 - Red arrow points to '💬 2': View comments

Hub
and dietary insights.

🔔 3 Publish Diet Post

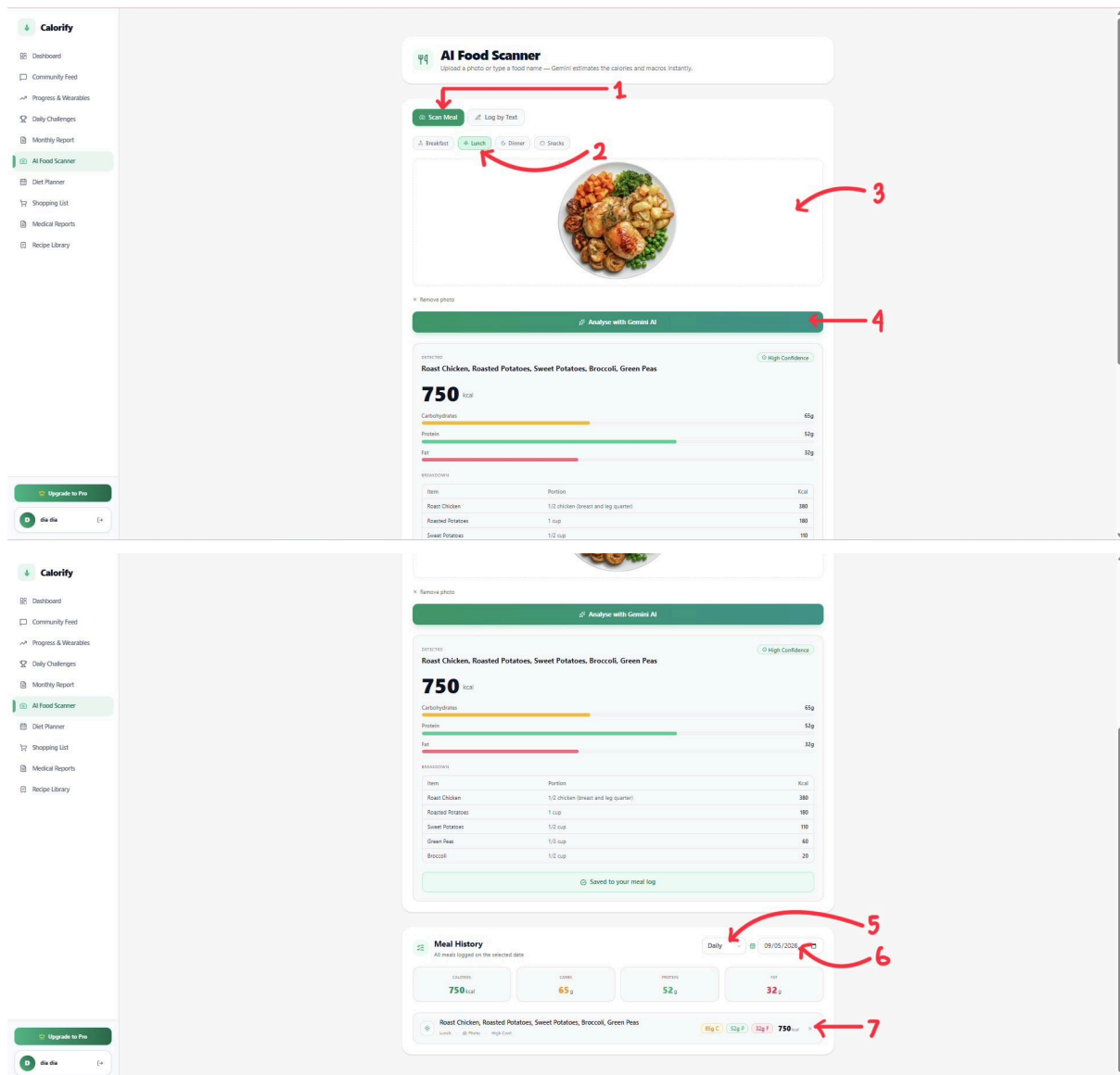
COMMUNITY ACTIVITY 3 updates

- Noorani commented: "heeeeee" 2d ago
Post: "hi"
- Noorani commented: "hey" 2d ago
Post: "hi"
- ♥ Noorani liked your post 6d ago
Post: "hi"

Sort: Newest First

Annotations:
 - Red arrow points to bell icon: Click the bell icon
 - Red arrow points to comment: you can check who liked and commented on your post

AI Image Meal Calorie Counter



1. Select to log calorie intake of a meal by uploading that meal image.
2. Select the type of meal.
3. Upload image of the meal from device. (For Mobile Phone, Select Camera, Upload from Gallery, Upload from file options are available)
4. Generate results of the estimated calorie intake.
5. Dropdown to select to display Daily/Weekly/Monthly Calorie Intake.
6. Select the day if Daily intake is selected from dropdown, Select the week if Weekly intake is selected from dropdown, Select the month if Monthly intake is selected from dropdown,
7. Cross button to delete the registered meal intake of that day/week/month.

AI Meal Calorie Counter (Text)

The screenshot shows the 'AI Food Scanner' interface within a 'Calorify' application. The interface includes a sidebar with navigation options like 'Dashboard', 'Community Feed', 'Progress & Wearables', 'Daily Challenges', 'Monthly Report', 'AI Food Scanner' (highlighted), 'Diet Planner', 'Shopping List', 'Medical Reports', and 'Recipe Library'. The main content area features a 'Log by Text' button (1), a meal type selector (2) with options for Breakfast, Lunch, Dinner, and Snacks, a 'FOOD NAME' input field (3) containing 'Beef Tahrir', a 'PORTION SIZE (optional)' input field (4) containing '1 medium plate', and an 'Estimate with Gemini AI' button (5). Below these inputs, the results for 'Beef Tahrir' are displayed, showing a total of 750 Kcal and a 'High Confidence' status. The results are broken down into macronutrients (Carbohydrates: 75g, Protein: 32g, Fat: 35g) and a detailed list of ingredients with their respective portions and calories.

Item	Portion	Kcal
Polao Rice	1.5 cups	360
Beef cubes	150g	250
Mustard oil and Ghee	2 tablepoons	150

1. Select to log calorie intake of a meal by text.
2. Select the type of meal.
3. Input the name of the meal.
4. Input the portion of meal intake.
5. Generate results of the estimated calorie intake.

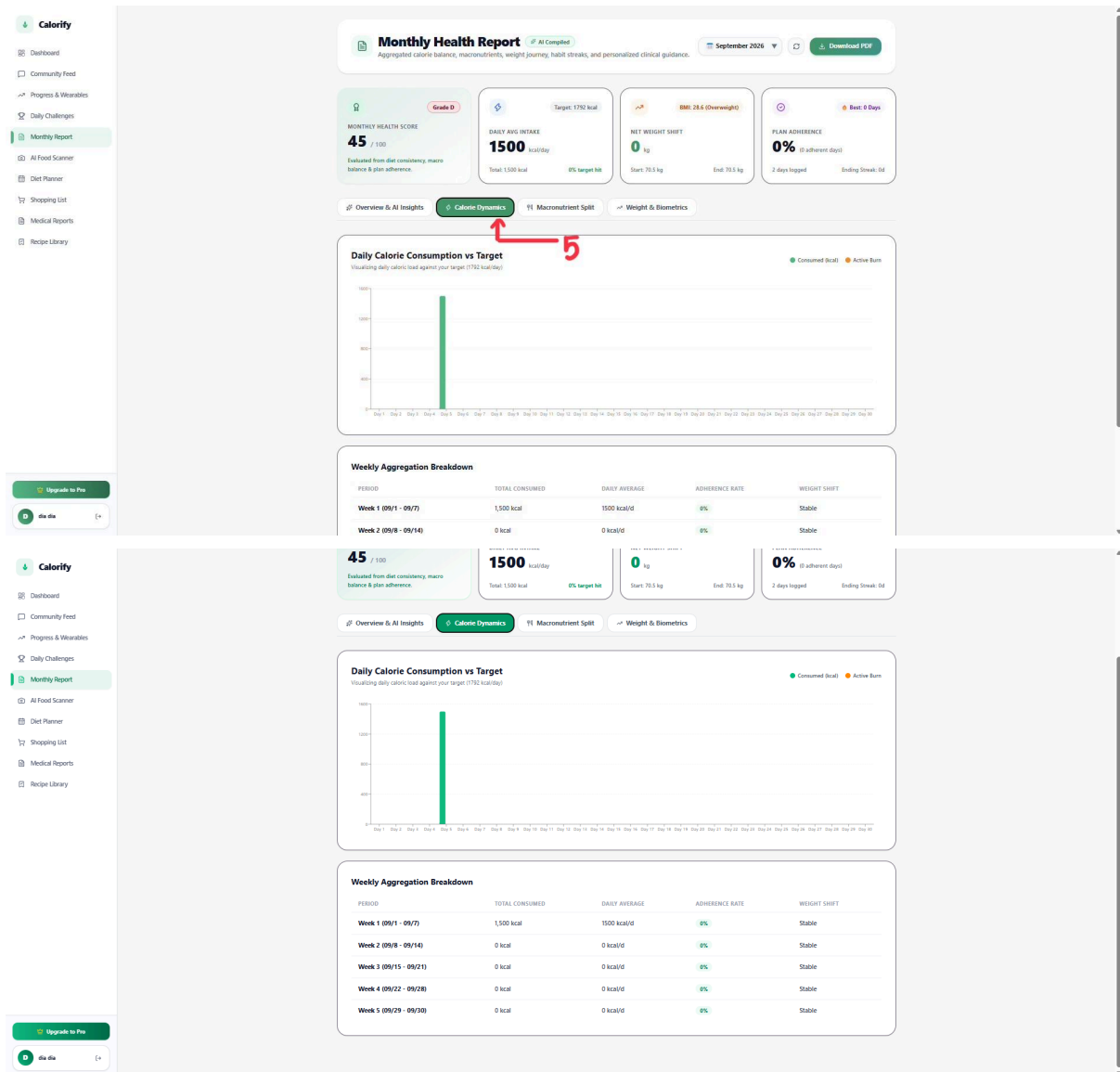
Monthly Report Generator

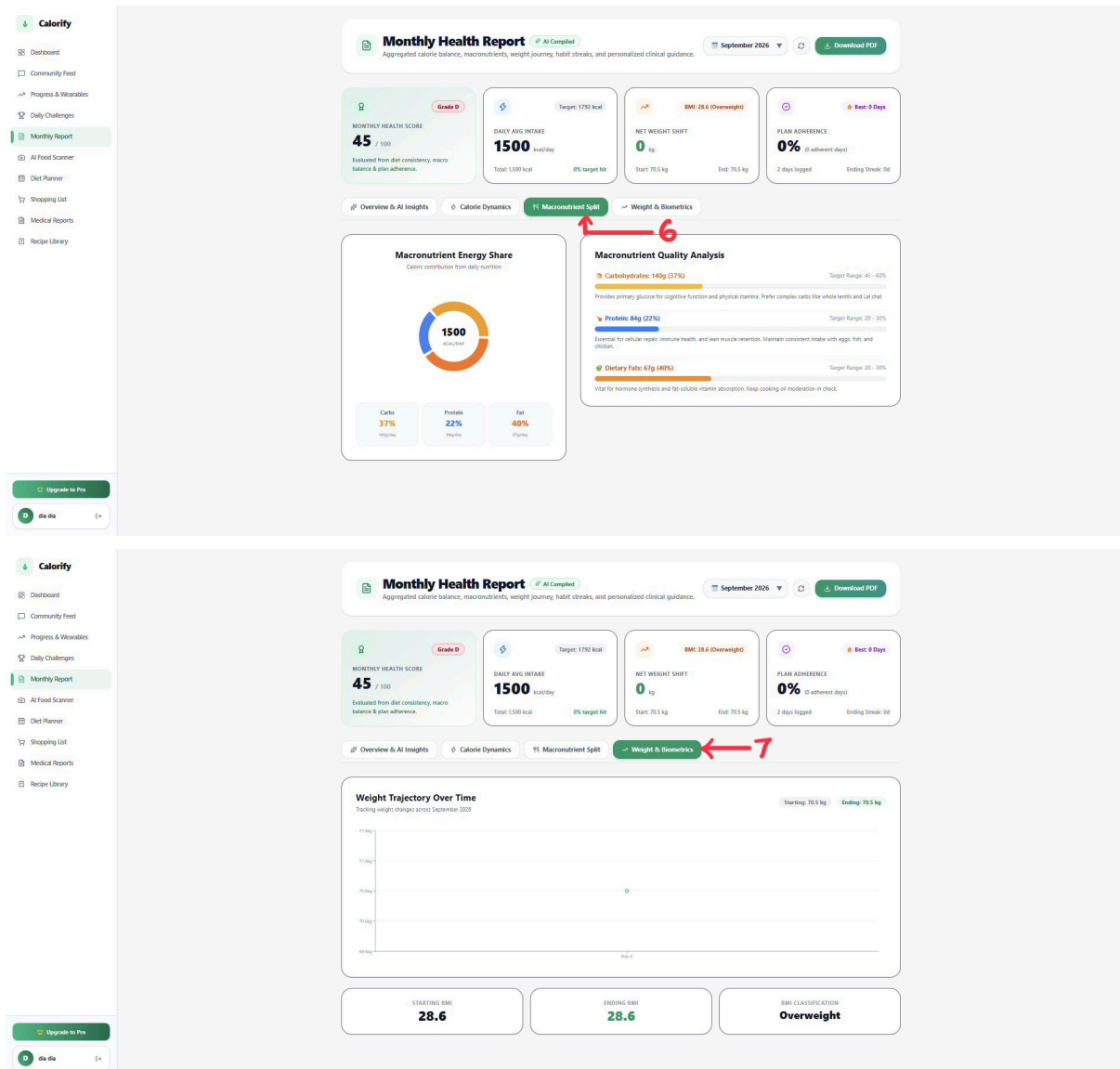
The image displays three sequential screenshots of the Calorify app's 'Monthly Health Report' for September 2026, illustrating a user's journey from data overview to personalized recommendations.

Screenshot 1 (Top): Shows the 'Monthly Health Report' overview. Key metrics include a Monthly Health Score of 45/100, a Daily Avg Intake of 1500 kcal/day, a Net Weight Shift of 0 kg, and a Plan Adherence of 0%. A 'Weight Trajectory Over Time' graph shows a starting BMI of 28.6 and an ending BMI of 28.6, classified as 'Overweight'. A red arrow points to the 'Download PDF' button in the top right corner.

Screenshot 2 (Middle): Shows the 'Overview & AI Insights' section. A red arrow points to the 'Overview & AI Insights' tab. The text states: 'Your data for September 2026 shows very low logging consistency with only 2 active days recorded, making it difficult to establish a stable routine for your weight maintenance goal. While your protein intake was moderate on logged days, the lack of physical activity and zero calorie-burn tracking require immediate attention to improve your overall health and metabolic wellness.'

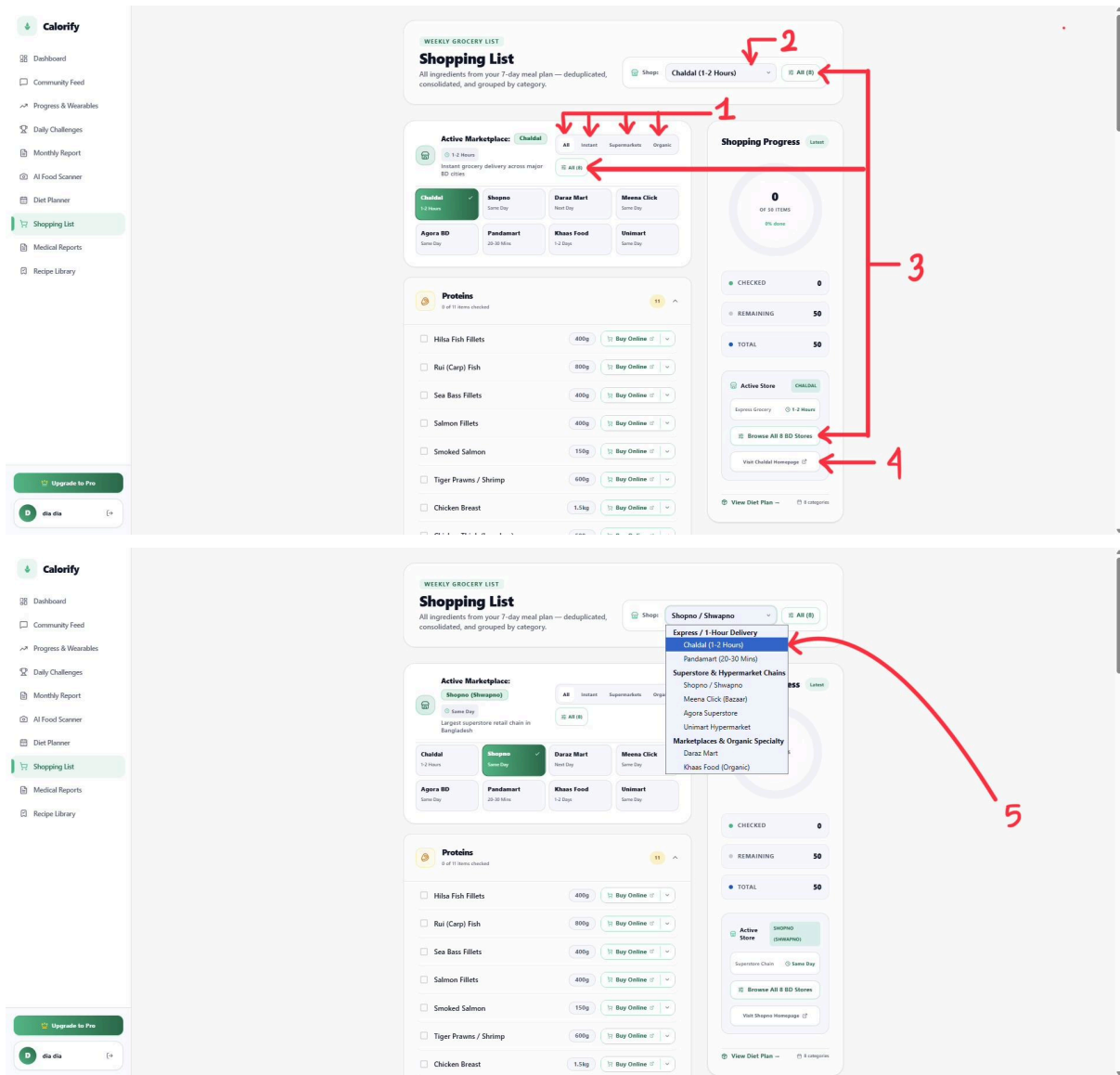
Screenshot 3 (Bottom): Shows the 'Tailored Bangladeshi Dietary Guidance' and 'Next Month's Focus' sections. A red arrow points to the 'Download Full PDF Report' button at the bottom right.

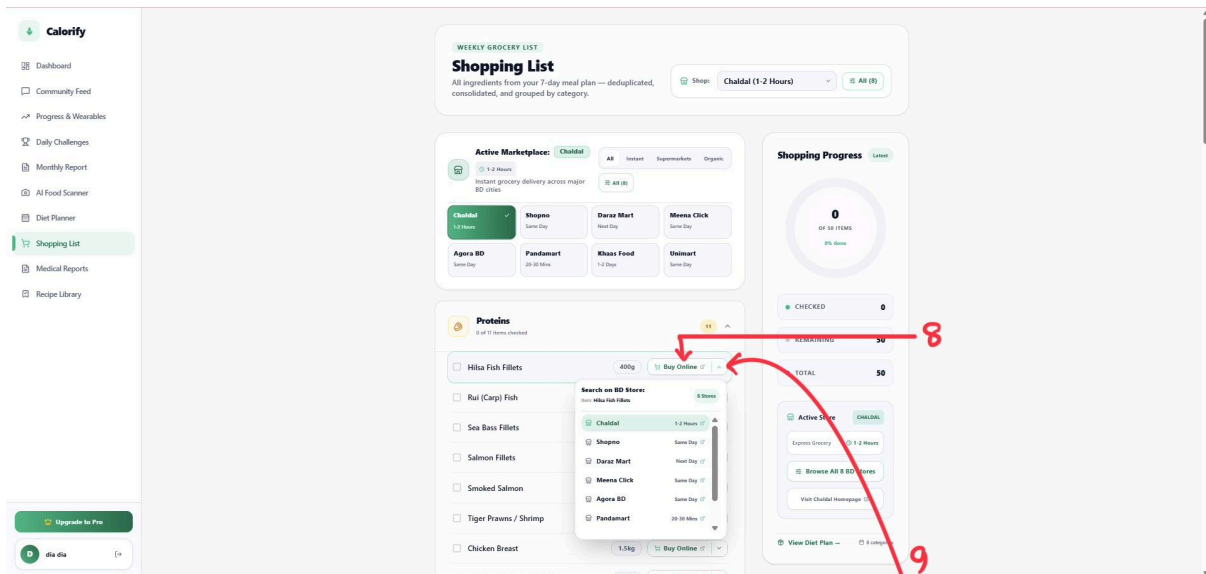
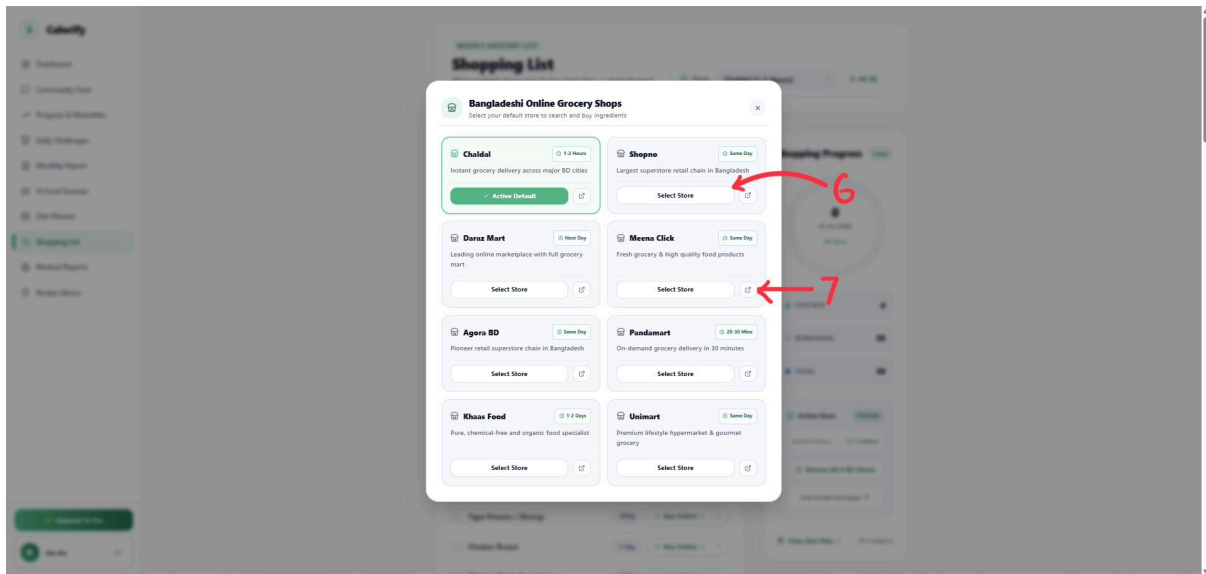




1. Displays all the months that meals were logged in by the user. Gives option to select which month's report to generate.
2. Downloads the generated monthly report locally.
3. Displays Personal Evaluation, Key Strength and Milestone, Areas of Optimization and Risks, Tailored Dietary Guidance, Next Month's Focus.
4. Downloads the generated monthly report locally.
5. Displays Calorie Consumptions vs Target Statistics, Weekly Aggregation Breakdown.
6. Displays Macronutrient Energy Share and Macronutrient Quality Analysis based on Monthly calorie intake.
7. Displays Weight Trajectory Over Time, Starting BMI, Ending BMI, BMI classification.
8. Regenerates the entire monthly report.

Online Marketplace Integration

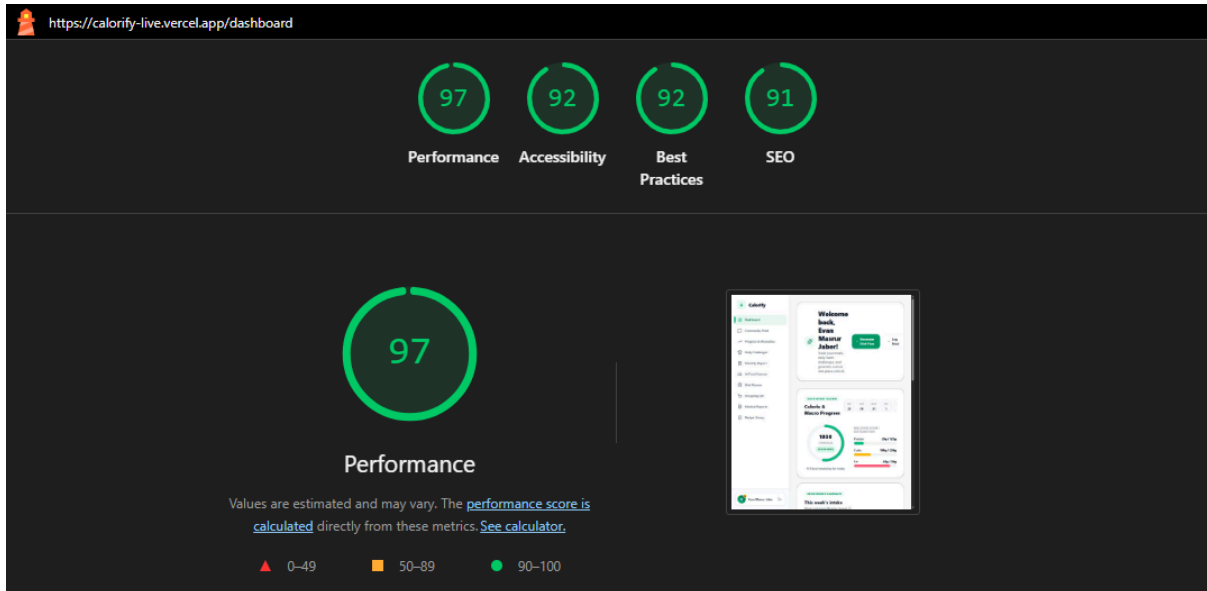




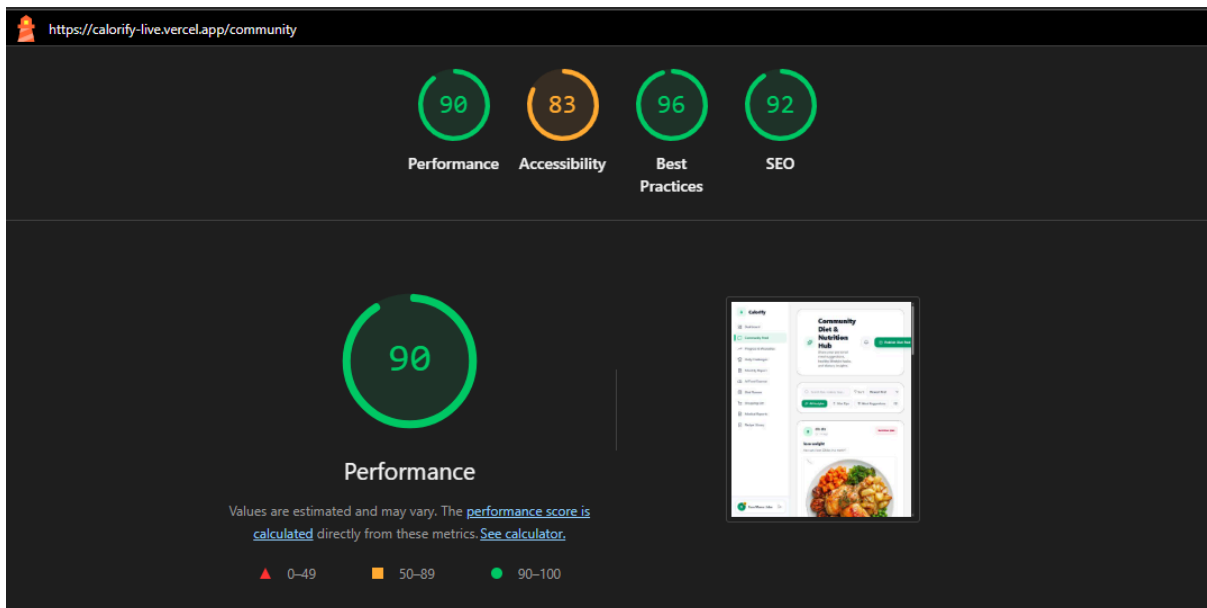
1. Displays options to choose categorational online shops based on user's diet preferences.
2. Clicking opens options to select Online Shops, displaying their delivery time.
3. Displays a separate blurred background tab of the available Online Shops to choose from.
4. Visit the site of the selected Online Shop.
5. Clicking selects an Online Shop to buy suggested groceries.
6. Clicking selects an Online Shop to buy suggested groceries.
7. Visit the site of the Selected Online Shop.
8. Visit the site of the Selected Online Shop searching for the corresponding grocery item.
9. The dropdown button displays and gives options to choose an Online Shop to buy that particular grocery item.

8. Performance and Network Analysis

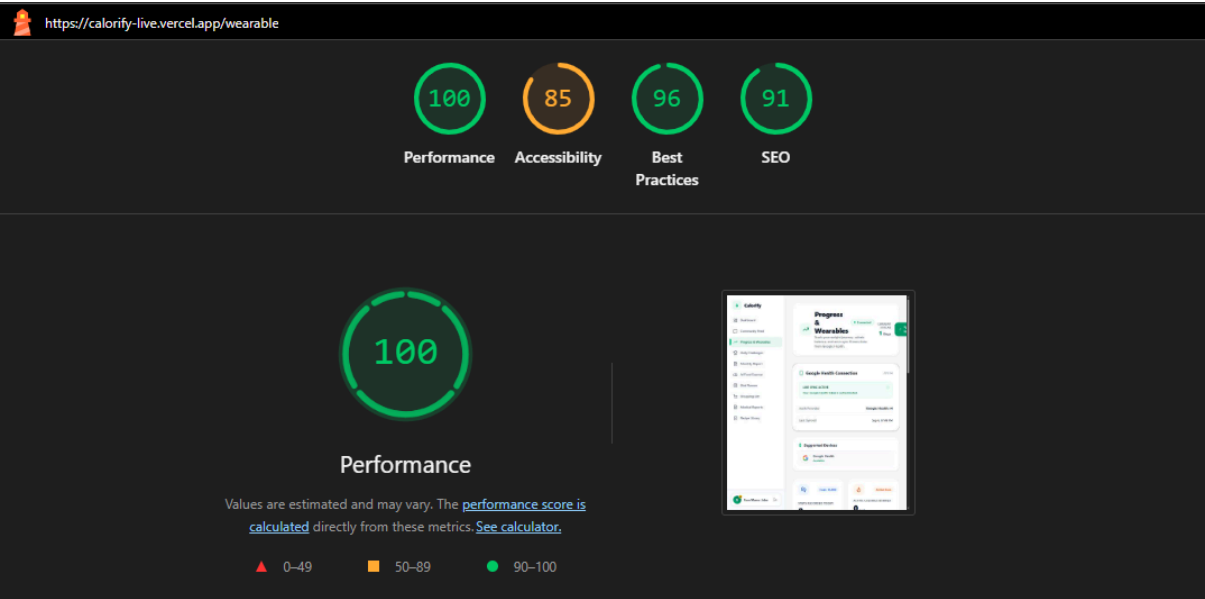
Lighthouse Reports (Dashboard page)



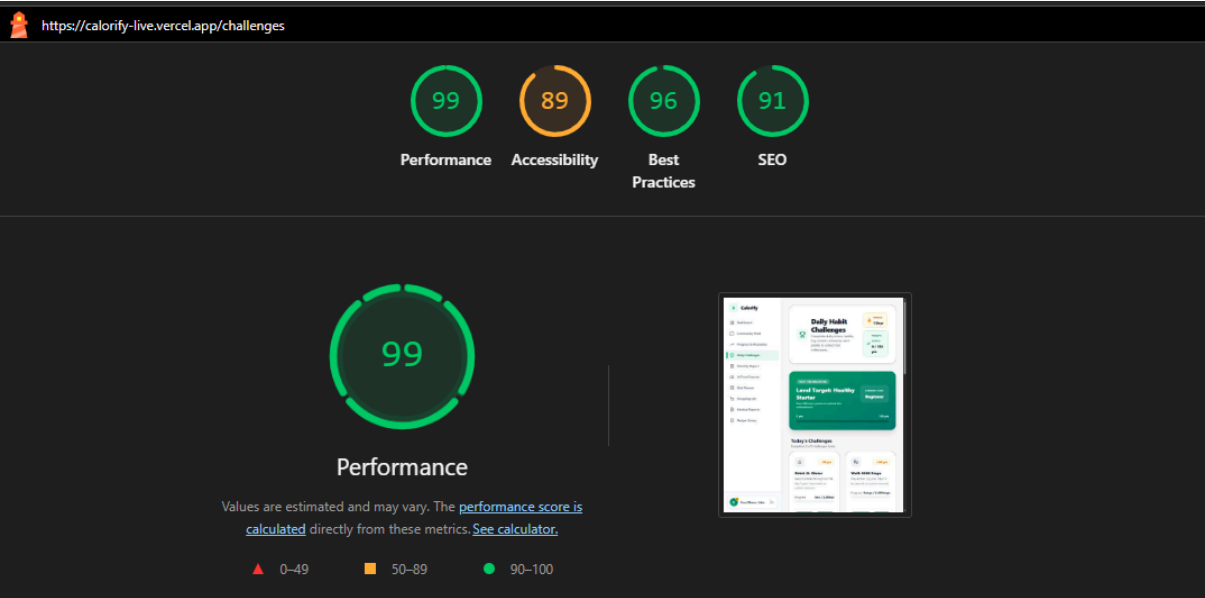
(Community Feed page)



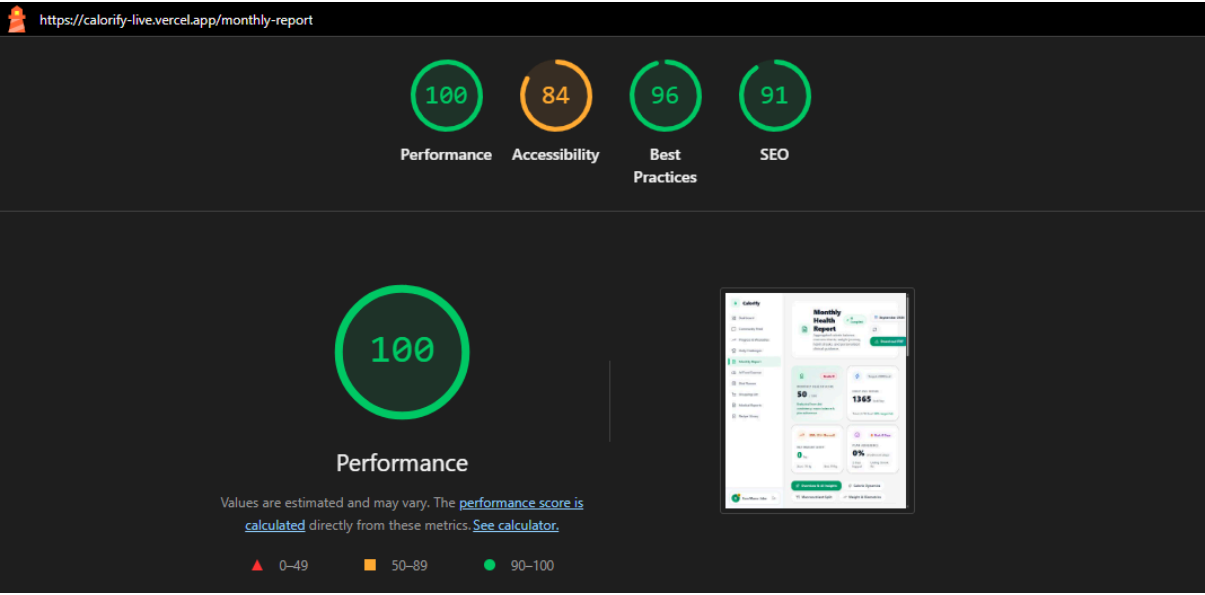
(Progress & Wearables page)



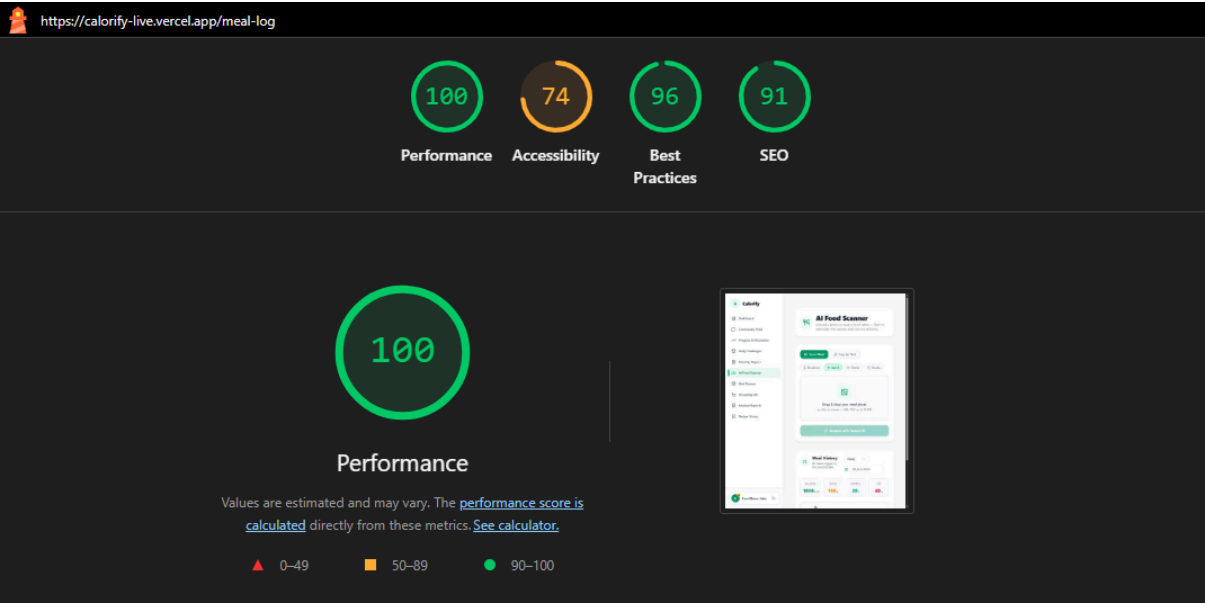
(Daily Challenges page)



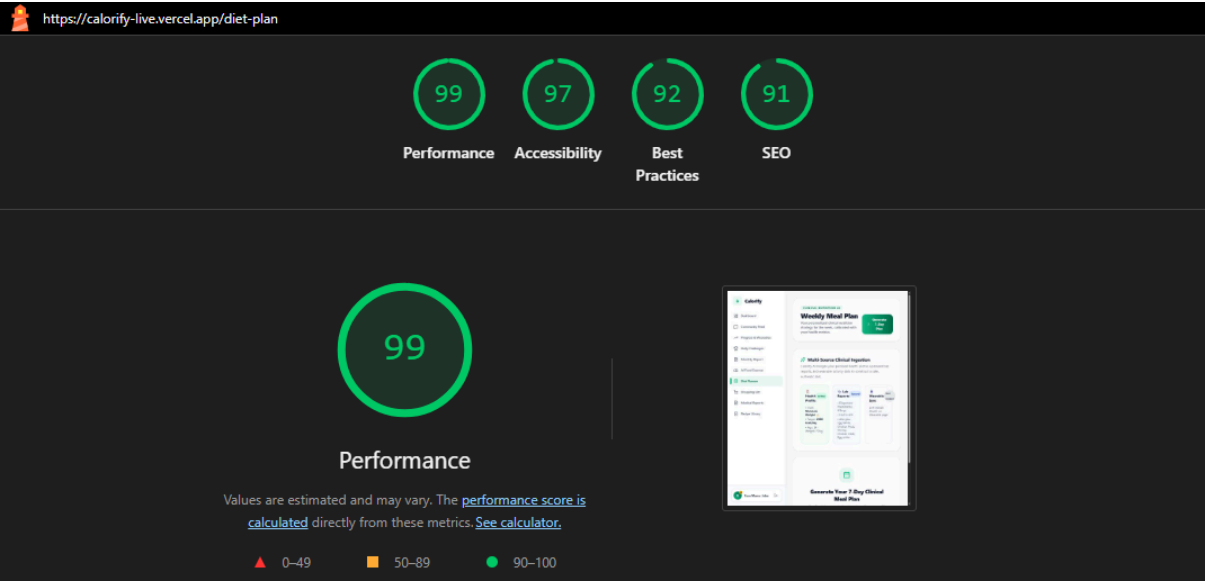
(Monthly Report page)



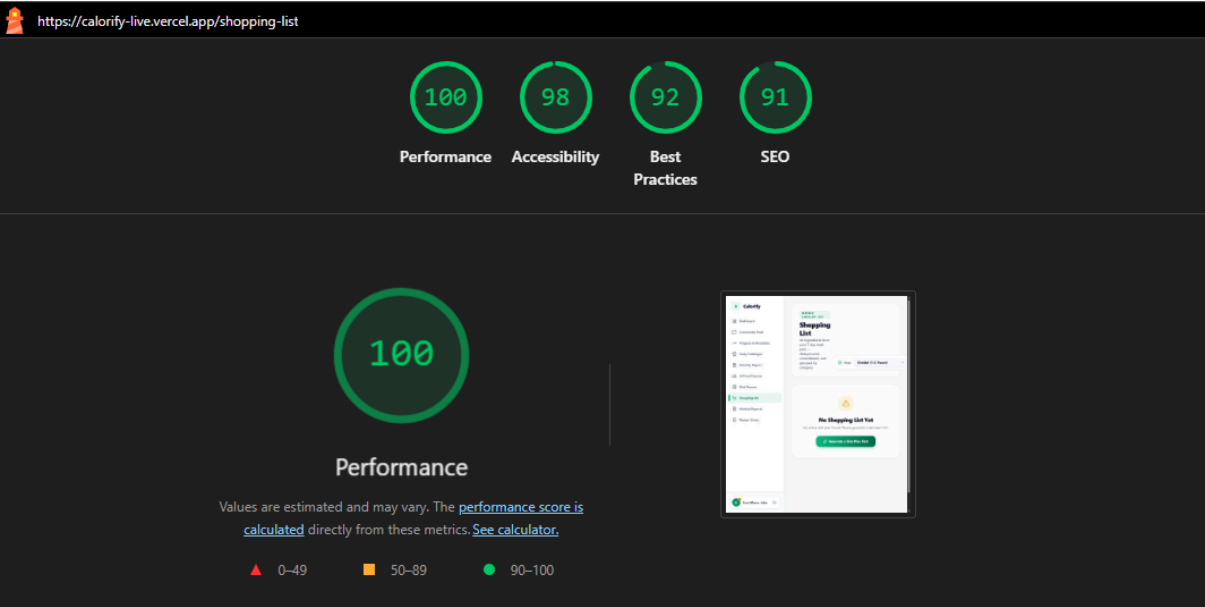
(AI Food Scanner page)



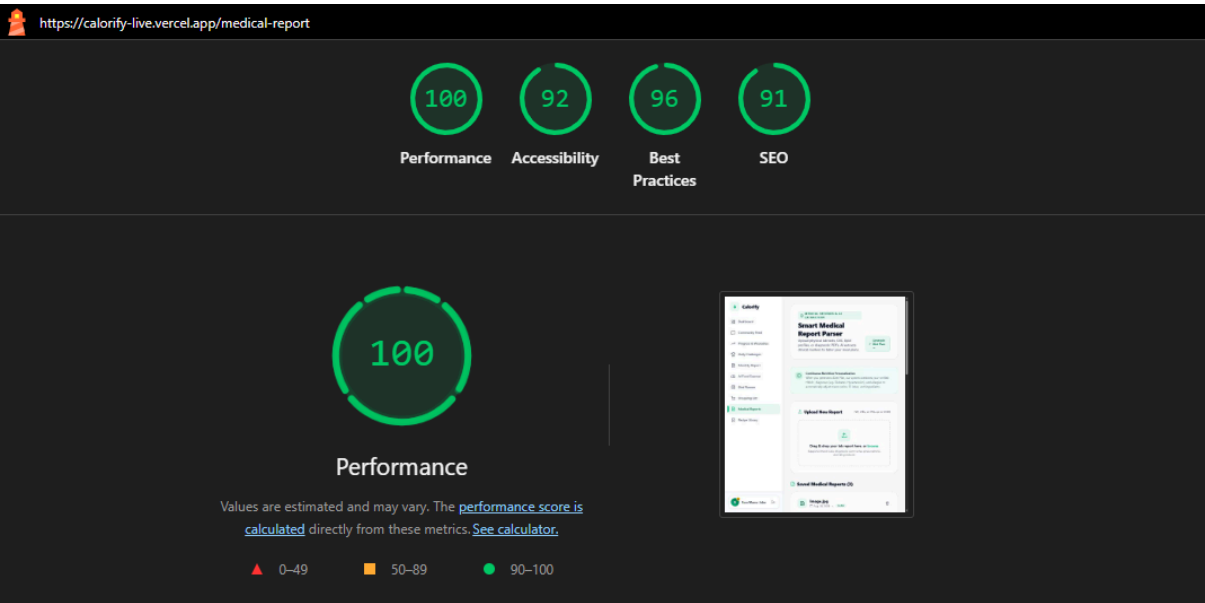
(Diet Generator page)



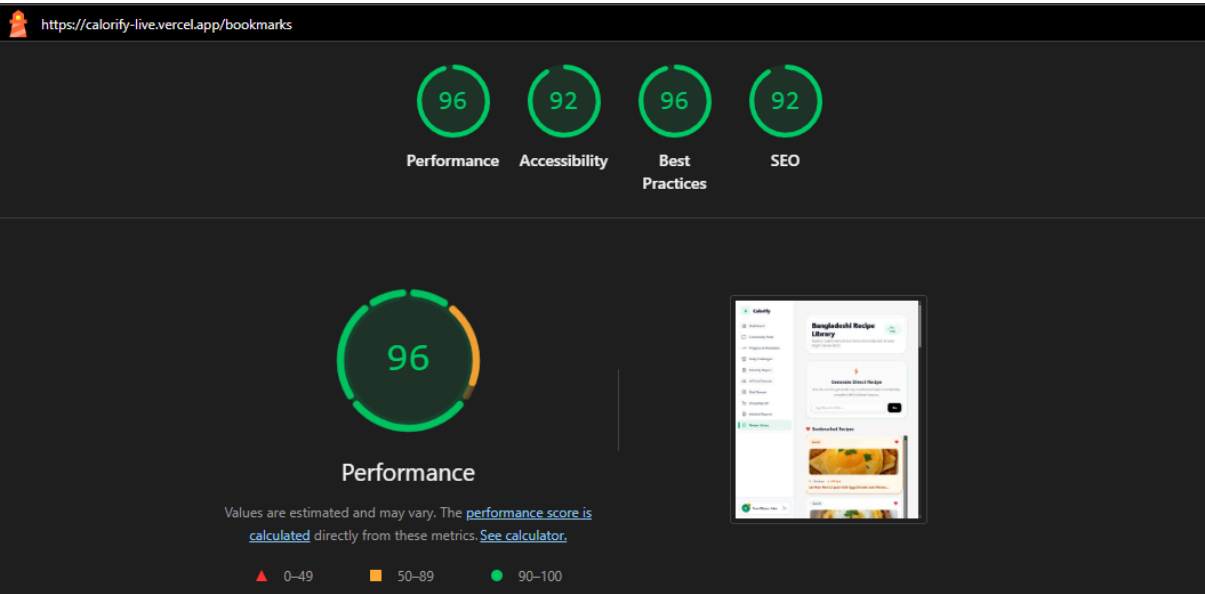
(Shopping List Generator page)



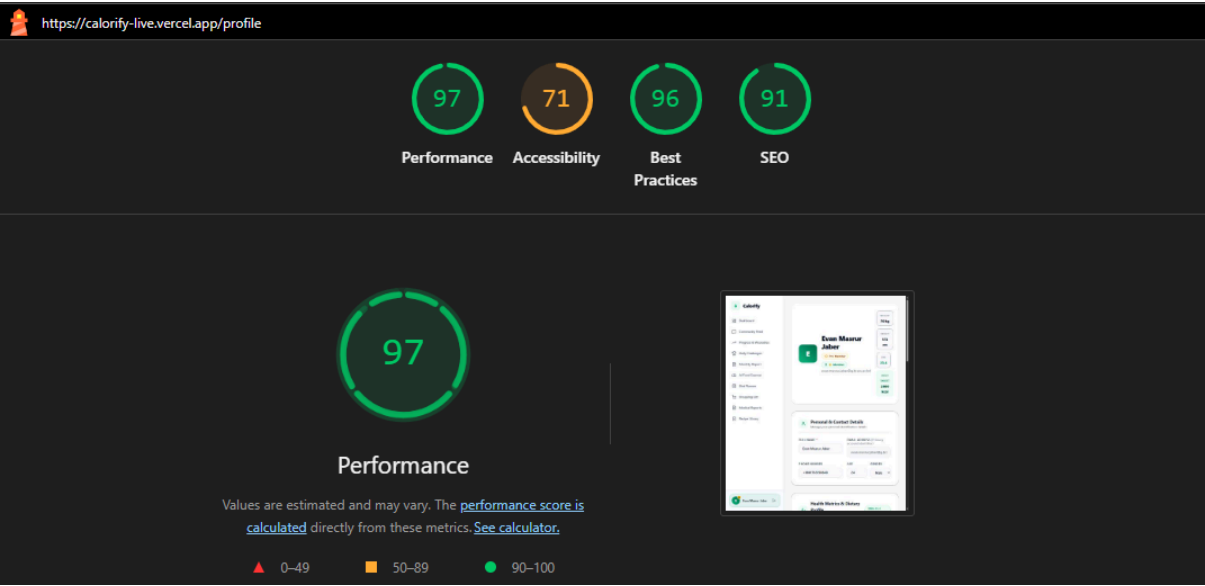
(Medical Report Parser page)



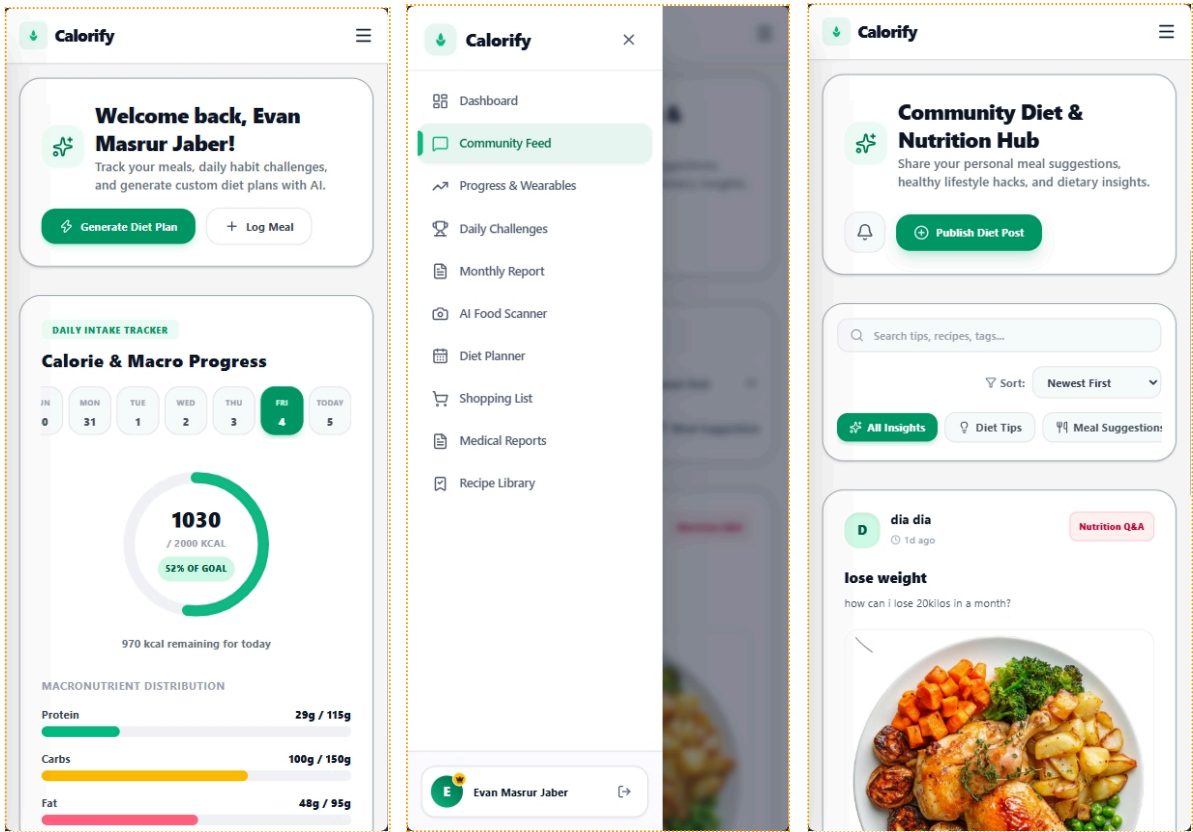
(Recipe Library page)

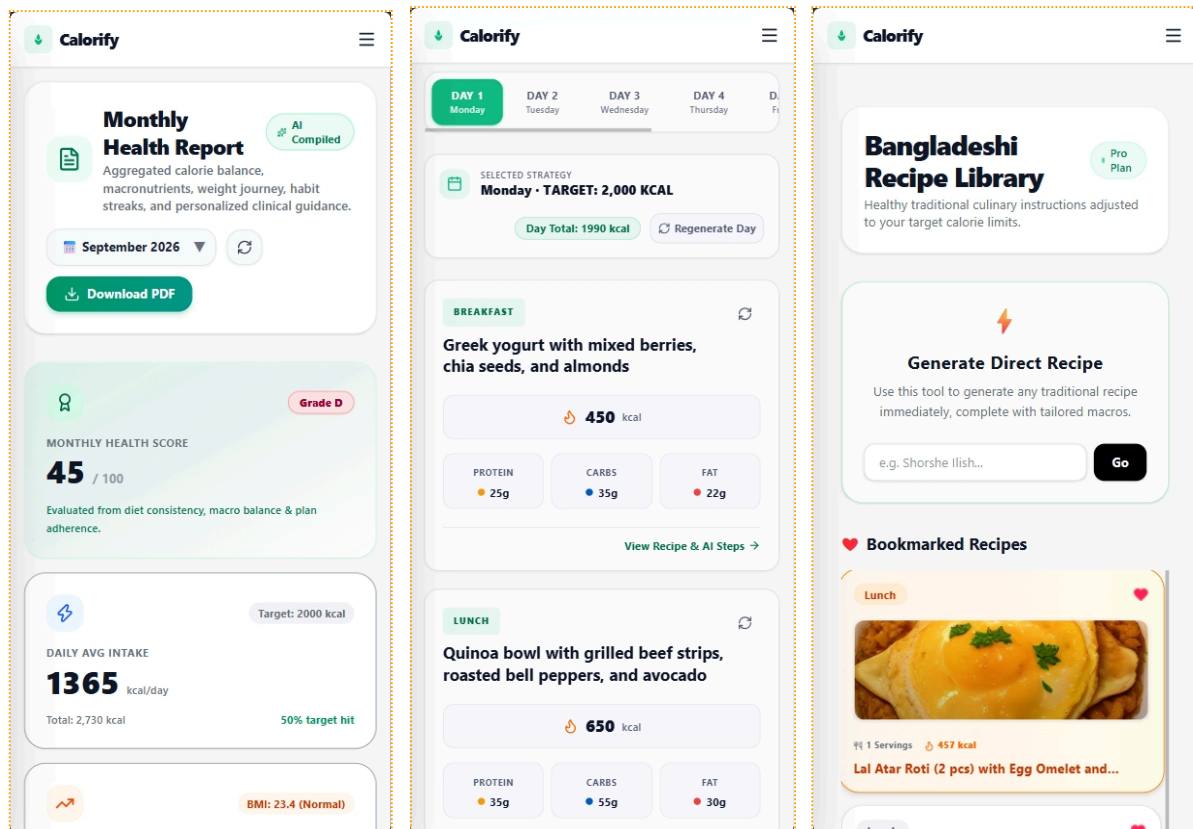


(Profile page)



System Screenshots in Mobile Viewport





9. Github Repo [Public] Link

[evanmasrurjaber/calorify-ai](https://github.com/evanmasrurjaber/calorify-ai)

10. Link of Deployed Project

<https://calorify-live.vercel.app>

11. Individual Contribution

Group member - 01	
Name: Evan Masrur Jaber	Student ID: 24341140
Functional Requirements which are developed by this member:	
1. Personalized Diet Plan Generator	
2. Automated Wearable Data Ingestion	
3. Ingredient Shopping List Generator	
4. Smart Medical Report Parser	

Group member - 02	
Name: Jarin Tasnim Dia	Student ID: 23101261
Functional Requirements which are developed by this member:	
1.AI Calorie Estimator (Image Upload)	
2.AI Meal Log	
3. Monthly Health Report Generator	
4. Online Marketplace Integration	

Group member - 03	
Name: Mohammed Mashrekin Yakub	Student ID: 23101253
Functional Requirements which are developed by this member:	
1. AI Recipe Generator	
2. Progress Tracker	
3. Pro Subscription & Payment Gateway	
4. Recipe Bookmarking	

Group member - 04	
Name: Noorani Faiza Khan	Student ID: 23301063
Functional Requirements which are developed by this member:	
1. Daily intake tracker	
2. Daily Habit Challenge	
3. Email Notification	
4. Community Feed	

12. References

N/A